Manual Técnico

Sistema de Gestión de Lavado Vehicular (Vehicle Washing)

Arquitectura de Microservicios, Modelo de Datos, Contratos de Integración, Seguridad y Lógica de Negocio

Brayan Estiven Patiño Cabrera (líder de proyecto), Juan Camilo Guilombo Osorio (líder de frontend) y Merari Urbano Quintero (líder de backend)

Tecnólogo en Análisis y Desarrollo de Software (ADSO), Servicio Nacional de Aprendizaje (SENA), Centro de Formación: Industria

Ficha 3145556

Instructora: Karol Daniela Correa

26 de octubre de 2026

Tipo de manual: Manual técnico (desarrolladores y arquitectos de software) · Versión 4.11

Control de Versiones

La Tabla 1 registra el historial de cambios de este documento. Cada modificación debe agregar una fila con la versión, la fecha, la descripción del cambio y el responsable. No se debe confundir con el control de versiones del código fuente, que se describe en la sección 15.

Tabla 1

Historial de Versiones del Manual Técnico

Versión	Fecha	Descripción del cambio	Responsable
1.0	2026-10-04	Creación inicial del manual técnico (en español) a partir de la documentación del proyecto.	Equipo de desarrollo
2.0	2026-10-05	Datos de portada completados, nombre del sistema ajustado a Vehicle Washing y cambio en la descripción de la estrategia de migraciones.	Equipo de desarrollo
3.0	2026-10-07	Traducción al inglés, formato APA (7.ª ed.) y verificación contra el código fuente. Se corrigieron: payment-service es .NET 8 (ADR-013); Spring Boot 4 en lugar de 3; SQL Server 2022; puertos reales; un changelog de Liquibase por servicio; número de tablas; el gateway no valida tokens; no hay bloqueo de cuenta; asignación manual de operarios (ADR-014); códigos de ejecución en inglés; solicitud real del endpoint de reservas; catálogo completo de endpoints y eventos; estructura real del proyecto; auditoría no poblada. Se agregaron dos diagramas reales.	Equipo de desarrollo
3.1	2026-10-09	Actualización contra el código de los once repositorios al 9 de octubre de 2026 (ADR-011, secciones 9 a 12, y ADR-015) y creación de esta versión en español (la versión en inglés es Manual_tecnico_LavadoVehicular_3145556_EN.md). Se actualizaron: 49 tablas (booking.schedule_history y notification.booking_tracking); puntos de fidelización por servicio y libro de puntos con reversión; promociones como cupones porcentuales; matriz de permisos de los roles fijos; interruptores de notificación del usuario; eventos, pruebas, estructura y contenedor de payment-service; avisos al personal; nuevos endpoints, rutas del gateway y códigos de error; nombre real del repositorio de operaciones; deuda técnica TD-002, TD-014, TD-015, TD-016 y TD-018 resuelta.	Equipo de desarrollo
4.0	2026-10-16	Revisión de los once repositorios con el sistema corriendo. Se actualizaron: 57 tablas con el diagrama entidad-relación generado desde la base de datos, diccionario de las tablas nuevas (bloqueo de cuenta, dirección, consentimiento, perfil del operario e inspección), asignación automática y elección del operario, bloqueo por intentos fallidos, informes de inspección en el servidor, datos del negocio, salud y correlación de payment-service, nuevos endpoints y eventos, resultados reales de las pruebas (639 unitarias y 19 de punta a punta), deuda técnica y discrepancias resueltas.	Equipo de desarrollo
4.1	2026-10-17	Revisión contra el código del 17 de octubre de 2026. Se documentó el informe de inspección de la aplicación móvil (sección 10.5), el cliente HTTP móvil con formularios multipart y la variable *EXPO_PUBLIC_WEB_URL* (sección 12.7), las dependencias *expo-image-picker* y *expo-image-manipulator* (Tabla 3) y la regla que ignora un 401 tardío después de cerrar sesión.	Equipo de desarrollo
4.2	2026-10-19	Integración continua con GitHub Actions en los 10 repositorios (Tabla 6) y cierre de TD-006.	Equipo de desarrollo
4.3	2026-10-19	Resultados de las pruebas del 19 de octubre de 2026 (655 automatizadas, 19 de punta a punta y 42 casos funcionales).	Equipo de desarrollo
4.4	2026-10-20	Revisión contra el código del 20 de octubre de 2026: el cambio de rol de una cuenta se revirtió en el código y el manual no lo documenta; se agregó la regla de que una cuenta de administrador no se desactiva (*ADMIN_CANNOT_BE_DISABLED*) y sus códigos de error 403; se aclara que la web y la app no usan la reprogramación del cliente ni el envío manual de notificaciones.	Equipo de desarrollo
4.5	2026-10-21	Revisión contra el código del 21 de octubre de 2026: el token de una sesión cerrada deja de servir (security-service y gateway, secciones 9.2, 13.2 y 14.1), cierre automático de las reservas vencidas (12.3 y Figura 3), candado de la asignación de operarios y conteo único de reservas sin operario (12.4), comprobante de máximo 2 MB y efectivo solo con su cuenta activa (12.5), cambios de la web y de la app (10.4, 10.5 y 12.7), pruebas del 21 de octubre (689 automatizadas y 19 de punta a punta) y diagnóstico. Se corrigieron además las rutas del gateway que faltaban en la Tabla 5 y la numeración de los changelogs (039 y semillas hasta la 112).	Equipo de desarrollo

Versión	Fecha	Descripción del cambio	Responsable
4.6	2026-10-21	Cambio de la regla de las reservas que terminan sin cerrarse, revisada por el equipo la noche del 21 de octubre: *ExpiredBookingsJob* solo cancela las programadas; operations-service (*FollowUpJob*, *FollowUpPolicy* y *FollowUpService*) pasa a NO_SHOW solo la confirmada sin operario, por la ruta interna */internal/v1/bookings/{id}/no-show*, y deja las demás en la lista "Por cerrar" del administrador (12.3, 12.4, 12.8 y Figura 3); nuevos endpoints, eventos ServiceNotStarted y ServiceNotFinished, tipos de notificación y changelogs 040 y 041 (11.5, 12.6, 13.3 y 13.4); conteo único de reservas sin operario también en la app móvil y demás cambios de la app (12.7); pruebas de la noche del 21 de octubre (707 automatizadas y 19 de punta a punta, 16.5); variables *FOLLOW_UP_** (16.2); diagnóstico y discrepancias (Tablas 26 y 29).	Equipo de desarrollo
4.7	2026-10-23	Revisión contra *develop* del 23 de octubre de 2026 y la versión *v1.0.0*: patrón MVVM de la web (10.4); capas, ViewModels y actualizaciones de la app móvil (APK obligatorio y EAS Update) (10.5); el operario inicia desde 15 minutos antes y sin límite posterior (Tabla de reglas de operations-service); **Por cerrar** revisa los últimos 28 días; el servidor calcula el monto del pago; las variables del 21 de octubre ya pasan por el compose (ACO-14 cerrada); las pruebas de punta a punta corren solo en local; control de versiones con *v1.0.0* y despliegue continuo preparado y sin probar (15.1 y 15.4); pruebas del 23 de octubre (825 y 13 de la app móvil) (16.5). Incluye el contenido de las ramas *fix/A-03*, *fix/A-05*, *fix/A-08*, *fix/A-15* y *fix/C-01* de Merari Urbano, revisado contra el código.	Equipo de desarrollo
4.8	2026-10-24	Revisión contra *develop* del 24 de octubre de 2026 (128 ramas fusionadas), que corre en el servidor público desde ese día: el gateway revisa la sesión con cualquier esquema, limita */api/v1/auth/** y fija tiempos de espera por ruta (componentes y 14.1); ventana para iniciar hasta 4 horas después del fin (reglas de operations-service); eventos *booking.status_changed* y *execution.operator_unassigned*, cola de errores de operations y entregas pendientes de notification, sin *booking.created* (Tabla 22); OWASP A07 aplicado; CI y despliegue continuo fusionado en *develop*, despliegue del 24 de octubre con *:prev*, *rollback.sh* y la protección contra bajar de versión (15.4); pruebas del 24 de octubre, 1 192 (16.5).	Equipo de desarrollo
4.9	2026-10-26	Pago al reservar, revisado contra *develop* del 26 de octubre de 2026 y desplegado en el servidor público ese día: booking-service consulta a payment-service por una ruta interna antes de cancelar (9.4.1 y 13.3) y consume *payment.reported* y *payment.confirmed* en la cola *booking.payment-events* (13.4); reglas de booking y payment (12.3 y 12.5); *paymentMode* en la creación de la reserva y códigos *INVALID_PAYMENT_MODE* y *CASH_BOOKING_LIMIT* (13.3); numeración de migraciones con la 049 y la semilla 117 (11.5 y 16.4); variables nuevas (16.2).	Equipo de desarrollo
4.10	2026-10-26	Versiones del 26 de octubre de 2026: *v1.1.0* en los 10 repositorios (primera versión de la app móvil) y *v1.1.1* en la web, la app móvil, payment-service e infra; desde *v1.1.1* de infra, payment-service también se despliega desde *main* (Tabla de tecnologías, 15.1 y 15.4).	Equipo de desarrollo
4.11	2026-10-26	Despliegue continuo comprobado: primera corrida automática del 26 de octubre por la noche, payment-service desde *main*, APK *versionCode* 6 y actualización por internet publicada tras comparar la huella nativa; arreglos de SonarCloud (15.4).	Equipo de desarrollo

Nota. Las fechas se escriben en formato ISO 8601 (año-mes-día). Elaborado por el equipo de desarrollo.

Tabla de Contenido

Control de Versiones 2
4. Introducción 6
4.1 Fuentes y Convenciones de Este Documento 6
5. Objetivo 6
5.1 Objetivo General 6
5.2 Objetivos Específicos 6
6. Alcance Técnico 7
6.1 Qué Cubre Este Manual 7
6.2 Qué No Cubre Este Manual 7
7. Descripción General del Sistema 7
7.1 Resumen de la Solución 7
7.2 Tipo de Aplicación y Componentes Principales 7
7.3 Restricciones de Arquitectura 9
8. Tecnologías Utilizadas 9
9. Arquitectura del Sistema 10
9.1 Estilo Arquitectónico 10
9.2 API Gateway 11
9.3 Arquitectura Interna de los Servicios 12
9.4 Comunicación entre Componentes 13
9.5 Aspectos Transversales 14
10. Estructura del Proyecto 15
10.1 Repositorios 15
10.2 Estructura de un Servicio Java 16
10.3 Estructura de payment-service 17
10.4 Estructura de la Aplicación Web 17
10.5 Estructura de la Aplicación Móvil 17
11. Base de Datos 19
11.1 Diagrama Entidad-Relación 19
11.2 Tablas Principales 19
11.3 Diccionario de Datos 21
11.4 Reglas de Integridad 28
11.5 Migraciones y Datos Iniciales 28
12. Módulos Técnicos 29
12.1 security-service 29
12.2 customer-service 29
12.3 booking-service 30
12.4 operations-service 31
12.5 payment-service 32
12.6 notification-service 32
12.7 Clientes Web y Móvil 33
12.8 Ciclos de Vida 34
13. API o Rutas del Sistema 34
13.1 Convenciones Generales 34
13.2 Autenticación en la API 35
13.3 Endpoints 35

13.4 Eventos de Dominio (RabbitMQ) 39
13.5 Salud y Verificación 40
14. Seguridad 40
14.1 Autenticación y Control de Sesiones 41
14.2 Autorización 41
14.3 Validación de Entradas 41
14.4 Secretos y Comunicación 41
14.5 Lista de Verificación OWASP Top 10 42
14.6 Protección de Datos Personales 42
14.7 Riesgos Aceptados 42
15. Control de Versiones 43
15.1 Código Fuente 43
15.2 Esquema de la Base de Datos 43
15.3 Decisiones de Arquitectura 43
15.4 Integración y Despliegue Continuos 44
16. Recomendaciones de Mantenimiento 45
16.1 Entorno Local 45
16.2 Configuración 45
16.3 Actualizaciones y Dependencias 45
16.4 Base de Datos 45
16.5 Pruebas 45
16.6 Diagnóstico de Problemas Frecuentes 47
16.7 Deuda Técnica 47
16.8 Evolución Recomendada 48
17. Glosario 48
Referencias 49
18. Anexos 49
Anexo A. Origen de las Figuras 50
Anexo B. Discrepancias Detectadas 51
Anexo C. Lista de Verificación para la Entrega 53

Nota. La numeración empieza en 4 porque las secciones 1 a 3 son la portada, el control de versiones y la tabla de contenido, que no llevan número en el título. Se conserva así para que sigan valiendo las referencias entre documentos por número de sección (por ejemplo, "Manual de Instalación, sección 10").

4. Introducción

El Sistema de Gestión de Lavado Vehicular (Vehicle Washing), llamado internamente *LavaRapido*, es una plataforma que permite reservar turnos de lavado, asignar operarios, ejecutar y calificar servicios, verificar pagos hechos por código QR y fidelizar a los clientes con puntos y cupones en un solo lavadero. Desde el punto de vista técnico, es un conjunto de seis microservicios de dominio (cinco escritos en Java 21 con Spring Boot 4 y uno en C# con .NET 8) detrás de un gateway, con una base de datos SQL Server 2022 compartida físicamente y separada lógicamente por esquemas, mensajería asíncrona mediante RabbitMQ, un cliente web en Angular 21 y un cliente móvil en React Native.

Este manual documenta cómo está construido el sistema: la arquitectura y las decisiones que la sustentan, la estructura del proyecto, el modelo de datos, la comunicación entre servicios, la API, la seguridad y las reglas de negocio. Está dirigido a desarrolladores, arquitectos, personal de soporte técnico e instructores que deban mantener o hacer evolucionar el sistema.

La arquitectura se apoya en conceptos ampliamente documentados: los microservicios (Newman, 2021), el diseño guiado por el dominio (Evans, 2004), la arquitectura hexagonal de puertos y adaptadores (Cockburn, 2005), el patrón de mensajería de publicación y suscripción (Hohpe y Woolf, 2003) y el estilo arquitectónico REST (Fielding, 2000).

4.1 Fuentes y Convenciones de Este Documento

La versión 3.0 se verificó contra el código fuente de security-service, customer-service, payment-service, lavarapido-infra, la aplicación web y la aplicación móvil (rama *develop*, 5 de octubre de 2026), y contra la documentación de los demás servicios (Equipo de Desarrollo Vehicle Washing, 2026a, 2026b, 2026c, 2026d, 2026e, 2026f). La versión 3.1 revisó los once repositorios (rama *develop*, 9 de octubre de 2026). La versión 4.0 volvió a revisar los once repositorios con el sistema completo corriendo en Docker (rama *develop*, 16 de octubre de 2026): las cifras de tablas, changelogs y pruebas salen de consultas a la base de datos y de la ejecución de las pruebas, no de documentos anteriores. La versión 4.5 se revisó contra el código del 21 de octubre de 2026. El Anexo B lista las discrepancias encontradas entre el código, la documentación y las versiones anteriores de los manuales, como exige la guía para elaborar manuales (Servicio Nacional de Aprendizaje [SENA], 2026). Los identificadores (esquemas, tablas, columnas, endpoints, eventos y clases) se escriben exactamente como en el código.

5. Objetivo

5.1 Objetivo General

Documentar la arquitectura, las tecnologías, la estructura, el modelo de datos, la comunicación entre servicios, la API, la seguridad y las reglas de negocio del Sistema de Gestión de Lavado Vehicular, de modo que otro desarrollador pueda entender, mantener y continuar el proyecto.

5.2 Objetivos Específicos

- Describir el estilo arquitectónico (microservicios con propiedad lógica de los datos) y la estructura interna de cada servicio.
- Presentar la estructura real de los repositorios y de las aplicaciones cliente.
- Presentar el modelo de datos de nueve esquemas, su diccionario y sus reglas de integridad.
- Documentar la comunicación síncrona (REST) y asíncrona (RabbitMQ) con sus endpoints y eventos.
- Explicar el modelo de seguridad: autenticación con JWT, autorización por rol y por propiedad, validaciones y protección de datos.
- Dejar recomendaciones de mantenimiento y pruebas, y registrar la deuda técnica y las discrepancias encontradas.

6. Alcance Técnico

6.1 Qué Cubre Este Manual

- Los seis servicios de backend (security, customer, booking, operations, payment y notification) y el API Gateway.
- La aplicación web (Angular) y la aplicación móvil (React Native con Expo) en su estructura y su relación con el backend.
- La base de datos SQL Server, sus migraciones con Liquibase y sus mecanismos de integridad.
- La mensajería con RabbitMQ, el correo mediante SMTP y las notificaciones push mediante Expo.
- El entorno local con Docker.

6.2 Qué No Cubre Este Manual

- El uso de la plataforma por parte de clientes, operarios y administradores (Manual de Usuario y Manual de Administrador).
- La instalación paso a paso (Manual de Instalación) y la puesta en operación de una versión (Procedimiento de Implementación).
- Los entornos de pruebas y de producción, que están fuera del alcance académico.
- Las pasarelas de pago, la facturación electrónica y la gestión de inventario, que no forman parte del sistema.

7. Descripción General del Sistema

7.1 Resumen de la Solución

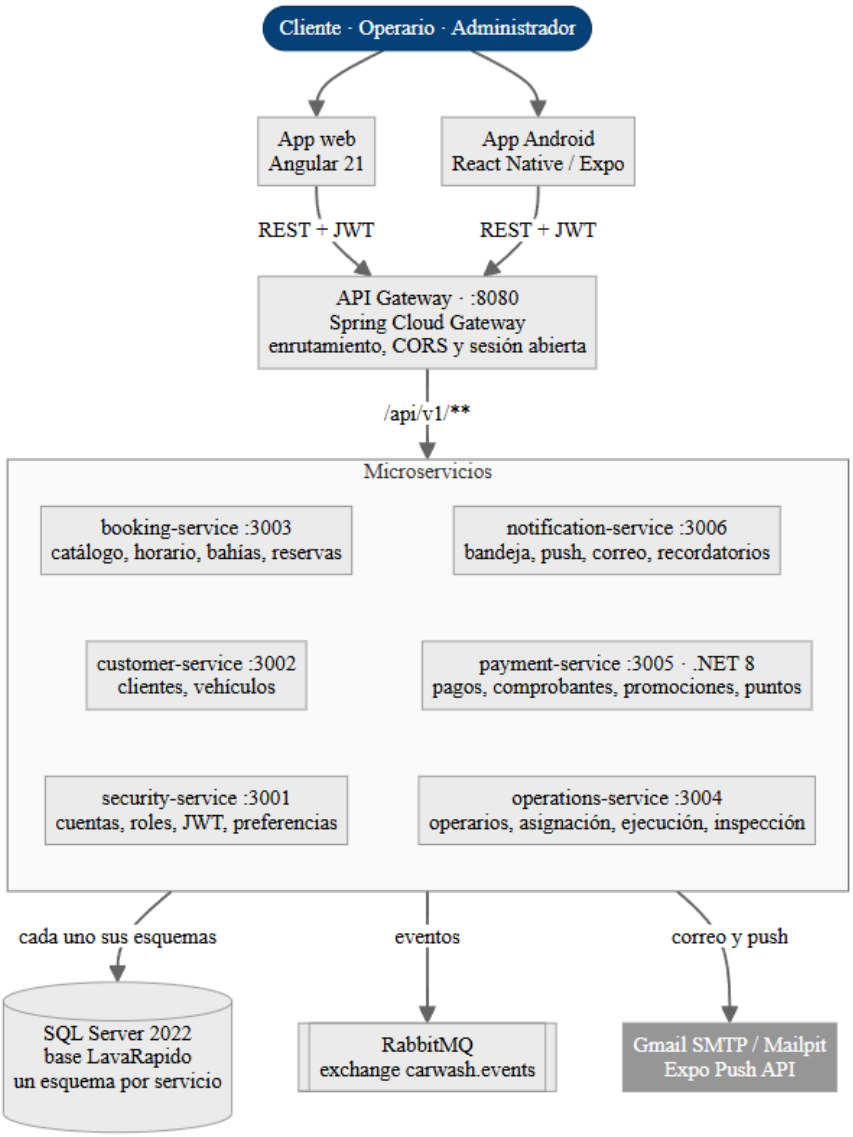
El sistema atiende a un solo lavadero. Interactúan con él tres roles fijos: el cliente, que registra vehículos, reserva turnos, paga, acumula puntos, canjea cupones y califica; el operario, que ejecuta los servicios asignados; y el administrador, que gestiona reservas, operarios, pagos, promociones, horarios y reportes. El pago no usa una pasarela externa: el cliente transfiere desde una aplicación bancaria y sube el comprobante, que el administrador aprueba o rechaza manualmente (ADR-008). No se recogen datos de tarjetas.

7.2 Tipo de Aplicación y Componentes Principales

Es un sistema distribuido cliente-servidor con un backend de microservicios (Figura 1 y Tabla 2).

Figura 1

Diagrama de Contenedores del Sistema



Nota. Elaborado por los autores a partir de *docker-compose.yml* y del catálogo de servicios. En desarrollo, la aplicación web llega al gateway mediante *proxy.conf.json*.

Tabla 2

Componentes Principales del Sistema

Componente	Tecnología	Puerto	Responsabilidad
Aplicación web	Angular 21	4200	Interfaz de los tres roles (cliente, operario y administrador).
Aplicación móvil	React Native 0.86 con Expo SDK 57	8081 (Metro)	Interfaz de los tres roles; notificaciones push y aviso de versión nueva (APK compilado con EAS).
api-gateway	Spring Cloud Gateway 2025.1 (webmvc)	8080	Punto de entrada único; enruta */api/v1/** por ruta; CORS; corta los tokens de sesiones cerradas (desde el 21 de octubre de 2026) con cualquier esquema del encabezado (desde el 24 de octubre); limita */api/v1/auth/** a 60 peticiones por minuto por IP y fija 5 s para conectar y 60 s para leer cada ruta (desde el 24 de octubre).
security-service	Java 21, Spring Boot 4	3001	Personas, cuentas, roles y permisos, sesiones, recuperación de la contraseña, perfil y preferencias.
customer-service	Java 21, Spring Boot 4	3002	Perfil del cliente, tipos de vehículo y vehículos.
booking-service	Java 21, Spring Boot 4	3003	Catálogo, precios y puntos, establecimiento, horario, bahías, disponibilidad y reservas.
operations-service	Java 21, Spring Boot 4	3004	Operarios, turnos, ausencias, asignación, ejecución y calificaciones.
payment-service	C#, .NET 8, ASP.NET Core, EF Core	3005	Cuentas de pago con QR, pagos con comprobante, revisión, reembolsos, pagos manuales, promociones (cupones) y libro de puntos de fidelización.
notification-service	Java 21, Spring Boot 4	3006	Bandeja, push, correo, recordatorios de reservas y avisos al personal.
SQL Server	SQL Server 2022 (contenedor)	1433	Base de datos *LavaRapido*, nueve esquemas, 57 tablas.
RabbitMQ	RabbitMQ 4 con administración	5672 y 15672	Eventos de dominio (exchange *carwash.events*).
Mailpit o SMTP	Mailpit (desarrollo) o Gmail	1025 y 8025	Correos.

Nota. Elaborado por los autores a partir de los archivos de construcción y de *docker-compose.yml*. El gateway es infraestructura y no se cuenta como servicio de dominio.

7.3 Restricciones de Arquitectura

- El sistema atiende a un solo lavadero.
- SQL Server es el único motor de base de datos; todos los servicios comparten la base *LavaRapido*, con un servicio dueño por esquema.
- Ningún servicio lee ni escribe las tablas de otro; no hay claves foráneas entre esquemas de servicios distintos (ADR-003).
- La verificación de los pagos es manual; no se usa pasarela de pagos (ADR-008).
- El entorno de ejecución es local con Docker.

8. Tecnologías Utilizadas

Tabla 3

Tecnologías y Herramientas del Proyecto

Componente	Tecnología	Versión o detalle
Lenguaje del backend (cinco servicios y gateway)	Java (Eclipse Temurin)	JDK 21

Componente	Tecnología	Versión o detalle
Framework del backend	Spring Boot	4.0.8: Web MVC, Validation, Security, OAuth2 Resource Server (JWT), Data JPA, Liquibase, Mail, AMQP, Actuator
Documentación de la API	springdoc-openapi	3.0.3 (Swagger UI solo en el perfil *dev*)
Gateway	Spring Cloud Gateway (server webmvc)	2025.1
Servicio de pagos	.NET, ASP.NET Core, EF Core (SQL Server), JwtBearer, Swashbuckle, RabbitMQ.Client	.NET 8; EF Core y JwtBearer 8.0.11
Base de datos	Microsoft SQL Server	2022 (imagen mssql/server:2022-latest); controlador mssql-jdbc
Migraciones	Liquibase	Un changelog por servicio (SQL formateado); imagen liquibase 4.29 para pagos
Mensajería	RabbitMQ	4 con complemento de administración; exchange de tipo topic *carwash.events*
Correo	Spring Mail con Mailpit (desarrollo) o Gmail	SMTP
Notificaciones push	Expo Push API	Requiere una versión EAS en el móvil
Frontend web	Angular, Angular Material y CDK, ngx-translate, Chart.js, jsPDF, xlsx, RxJS	21.2.25, 21.2.14, 17, 4.5, 4.2, 0.20, 7.8; TypeScript 5.9
Frontend móvil	React Native, Expo, React, React Navigation, i18next; expo-image-picker y expo-image-manipulator (fotos de la inspección)	0.86.3, SDK 57, 19.2, 7, 26; 57.0; TypeScript 6.0
Contenedores	Docker y Docker Compose	Docker 24 o posterior; Compose v2
Pruebas	JUnit 5 y Spring Boot Test (Java), xUnit (.NET), Vitest (web), Jest con *jest-expo* (móvil, desde el 23 de octubre de 2026)	Mediante el Maven Wrapper, *dotnet test*, *ng test* y *npm test*
Control de versiones	Git y GitHub	Ramas *main* (versiones etiquetadas: *v1.0.0* en 8 repositorios el 23 de octubre de 2026 y en payment-service el 24; *v1.1.0* en los 10 repositorios el 26 de octubre, y *v1.1.1* en la web, la app móvil, payment-service e infra ese mismo día), *develop*, *feature/** y *fix/**
Integración continua	GitHub Actions	`ci.yml` en los 10 repositorios: compila y corre las pruebas en cada push o pull request a *develop* y *main*; `lavarapido-infra` tiene además el flujo manual "Desplegar en Azure" (sin ejecutar todavía) y el despliegue continuo preparado en una rama sin fusionar (sección 15.4)
Diagramas y contratos	Mermaid (C4 y diagramas UML), OpenAPI 3	Carpetas 08-uml y 07-api de la documentación

Nota. Versiones tomadas de *pom.xml*, *Directory.Build.props*, los archivos *.csproj* y *package.json* de cada repositorio.

9. Arquitectura del Sistema

9.1 Estilo Arquitectónico

El backend adopta una arquitectura de microservicios con propiedad lógica de los datos: cada servicio se despliega de forma independiente, se organiza alrededor de una capacidad del negocio y es dueño de sus datos (Newman, 2021). Por razones académicas, los seis servicios comparten la misma instancia y la misma base de datos de SQL Server, pero cada esquema tiene exactamente un servicio dueño y ningún servicio escribe ni consulta el esquema de otro (ADR-003). La descomposición actual en seis servicios reemplazó una anterior con más servicios (ADR-002, reemplazado por ADR-009).

Tabla 4

Servicios, Esquemas Propios y Tablas

Servicio	Esquemas	Tablas	Profundidad del dominio
security-service	security, audit	12	Media: autenticación, bloqueo por intentos fallidos, roles, permisos, reglas de contraseña, preferencias y consentimiento.
customer-service	customer	3	Media: propiedad de clientes y vehículos.
booking-service	catalog, booking	13	Alta: disponibilidad, superposición de bahías, ciclo de vida, precios inmutables, puntos por servicio e historial del horario.
operations-service	execution	11	Alta: turnos, ausencias, asignación manual y automática, ejecución, calificación, perfil del operario e informes de inspección.
payment-service	payment, promotion	12	Alta: revisión de pagos, reembolsos, cupones porcentuales y libro de puntos.
notification-service	notification	6	Media: bandeja, consumo idempotente, push, correo, recordatorios y seguimiento de reservas para los avisos al personal.

Nota. Total: 57 tablas, comprobado con una consulta a *sys.tables* el 16 de octubre de 2026. Fuente: changelogs de cada servicio.

9.2 API Gateway

Los clientes web y móvil llaman solo al gateway (puerto 8080), de modo que no necesitan conocer las direcciones internas (ADR-005). El gateway enruta */api/v1/** por ruta hacia el servicio dueño, reenvía sin cambios el método, el cuerpo, la cadena de consulta y los encabezados *Authorization*, *X-Correlation-Id* y *Accept-Language*, responde CORS (incluida la verificación previa) y devuelve 404 para */internal/** y para las rutas desconocidas. **No verifica la firma de los tokens**: cada servicio verifica el JWT por sí mismo (ADR-006). Desde el 21 de octubre de 2026 sí confirma que la sesión del token siga abierta: *SessionCheckFilter*, que corre justo después del CORS, pregunta a security-service con *GET /api/v1/auth/session* y el mismo encabezado *Authorization*; si la respuesta es 401, devuelve ese 401 sin pasar la petición. Guarda cada respuesta 30 segundos por huella SHA-256 del token (nunca el token), olvida la de un token cuando pasa por *POST /api/v1/auth/logout* y, si security-service no contesta en 3 segundos o responde 5xx, deja pasar la petición para que una caída de security-service no tumbe el resto de la API. No revisa las rutas públicas, las de */api/v1/auth/* (salvo el logout) ni las solicitudes sin token. Se configura con *SESSION_CHECK_ENABLED*, *SESSION_CHECK_CACHE_TTL* y *SESSION_CHECK_TIMEOUT*. No tiene base de datos ni límite de solicitudes. La Tabla 5 muestra las rutas.

Tabla 5

Rutas del API Gateway

Prefijos de ruta	Servicio
/api/v1/auth/*, */api/v1/users/**, /api/v1/admin/users/*, */api/v1/admin/roles-permissions/**	security-service :3001
/api/v1/vehicles/**, /api/v1/vehicle-types, /api/v1/admin/vehicles	customer-service :3002
/api/v1/catalog/*, */api/v1/schedule/**, /api/v1/establishment, /api/v1/bookings/*, */api/v1/admin/bookings/**, /api/v1/admin/catalog/*, */api/v1/admin/schedule/**, /api/v1/admin/bays/*, */api/v1/admin/establishment, /api/v1/operator/bookings/**	booking-service :3003
/api/v1/admin/operators/*, */api/v1/operators/**, /api/v1/admin/inspections/*, */api/v1/public/inspections/**, /api/v1/admin/assignments/*, */api/v1/operator/services/**, /api/v1/operator/ratings, /api/v1/ratings/**	operations-service :3004
/api/v1/payments/*, */api/v1/payment-accounts, /api/v1/admin/payments/**, /api/v1/admin/payment-accounts/*, */api/v1/admin/payment-methods, /api/v1/admin/promotions/**, /api/v1/loyalty/**	payment-service :3005
/api/v1/notifications/*, */api/v1/admin/notifications/**	notification-service :3006

Nota. Las direcciones se configuran con ROUTE_SECURITY_URL, ROUTE_CUSTOMER_URL, ROUTE_BOOKING_URL, ROUTE_OPERATIONS_URL, ROUTE_PAYMENT_URL y ROUTE_NOTIFICATION_URL; el archivo de Compose las define todas con los nombres de los contenedores.

9.3 Arquitectura Interna de los Servicios

Servicios Java: arquitectura hexagonal. Los cinco servicios Java siguen el estilo de puertos y adaptadores (Cockburn, 2005; ADR-007). El dominio no depende de Spring, JPA, SQL Server ni RabbitMQ; las dependencias apuntan hacia el dominio (Tabla 6).

Tabla 6

Capas de los Servicios Java

Capa	Contenido	Regla
domain	model (entidades y objetos de valor), service (políticas del dominio), event, exception (con un código estable), port/in (interfaces de casos de uso), port/out (repositorios, publicadores, directorios).	No importa Spring ni JPA.
application	usecase: servicios anotados con @Service y @Transactional que implementan los puertos de entrada.	Coordina puertos; las reglas pertenecen al dominio.
infrastructure	adapter/in/web (controladores y DTO), adapter/out/persistence (entidades y repositorios JPA), adapter/out/messaging (RabbitMQ), adapter/out/security y notification, config (seguridad, CORS, JWT, OpenAPI, correlación).	Implementa los puertos de salida; controladores delgados.

Nota. Estructura verificada en los cinco servicios Java.

El siguiente fragmento de *AuthenticationService* (security-service) muestra un caso de uso que solo coordina puertos: valida la contraseña, aplica la regla del dominio (*ensureCanAuthenticate*), registra la sesión y emite el token.

```
@Service
@Transactional
public class AuthenticationService implements AuthenticateUserUseCase, LogoutUseCase {
    @Override
    public AuthenticationResult login(LoginCommand command) {
        UserAccount account = findByEmail(command.email())
                .filter(found -> passwordHasher.matches(command.password(), found.passwordHash()))
                .orElseThrow(InvalidCredentialsException::new);
        account.ensureCanAuthenticate();          // una cuenta desactivada no puede ingresar
        Instant now = clock.instant();
        account.recordSuccessfulLogin(now);
        accounts.save(account);
        long sessionId = sessions.open(account.id(), now, now.plus(tokenIssuer.timeToLive()),
                command.ipAddress(), command.userAgent());
        IssuedToken token = tokenIssuer.issue(account, sessionId, now);
        return new AuthenticationResult(token.value(), token.expiresAt(), UserAccountView.from(account));
    }
}
```

Nota. Fragmento abreviado de la clase. Cuando el correo no existe, el servicio compara contra un hash ficticio para que el tiempo de respuesta no revele qué cuentas existen.

payment-service: arquitectura por capas (Clean) en .NET. Se organiza en cuatro proyectos con dependencias hacia el dominio (ADR-013): *PaymentService.Domain* (agregado *Payment* con su máquina de estados, *PaymentAccount*, *PaymentReceipt*, *PaymentPolicy*, *Promotion*, *LoyaltyTransaction* y eventos de dominio, sin paquetes NuGet), *PaymentService.Application* (casos de uso por funcionalidad: *PaymentCommandService*, *PaymentQueryService*, *BookingPaymentGuard* y *AmountDueCalculator*; *PaymentAccountService*; *PromotionAdminService* y *CustomerPromotionService*; *LoyaltyService* y *LoyaltyRewardService*; puertos de entrada y salida en *Ports/In* y *Ports/Out*, incluidos *IBookingDirectory* e *IDomainEventPublisher*), *PaymentService.Infrastructure*

(repositorios EF Core, *BookingServiceDirectory*, un cliente HTTP hacia booking-service, y el publicador de RabbitMQ en *Messaging*) y *PaymentService.Api* (controladores, JWT bearer, CORS, *GlobalExceptionHandler*, *EnvFile* y Swagger). Mantiene el mismo contrato que los servicios Java: tokens HS256 de security-service, errores RFC 9457 con un código estable, */actuator/health* y configuración desde *lavarapido-infra/.env*.

9.4 Comunicación entre Componentes

9.4.1 Comunicación Síncrona con REST

REST se usa cuando quien llama necesita una respuesta para continuar (ADR-004). Las llamadas entre servicios no pasan por el gateway: quien llama reenvía el token del usuario, de modo que el servicio destino aplique sus propias reglas de acceso (Tabla 7).

Tabla 7

Llamadas Síncronas entre Servicios

Origen	Destino y llamada	Motivo
customer-service	security-service GET /api/v1/users/me	Leer el personId para crear un perfil de cliente faltante.
booking-service	customer-service GET /vehicles/{id}, /vehicles, /admin/vehicles	Propiedad y tipo del vehículo (ADR-012).
operations-service	booking-service /admin/bookings, /operator/bookings, PATCH /operator/bookings/{id}/status, /bookings/me; /internal/v1/bookings, /internal/v1/bookings/{id} y POST /internal/v1/bookings/{id}/no-show (llave interna)	Reservas por asignar, con su bahía; pasarlas a IN_PROGRESS o COMPLETED; validar la reserva que se califica; leer la reserva cuando llega BookingConfirmed para la asignación automática, sin token de usuario (ADR-014); cerrar como NO_SHOW la confirmada que terminó sin operario (*FollowUpJob*).
operations-service	security-service GET /admin/users?role=OPERATOR; GET /internal/v1/users/{id}/contact (llave interna)	Cuentas y nombres de los operarios; nombre del cliente de cada servicio y estado de la cuenta (una cuenta que no existe responde OPERATOR_NOT_FOUND).
payment-service	booking-service GET /bookings/{id}, /bookings/me, /admin/bookings/{id}	Total, estado, dueño y puntos de fidelización de la reserva; el navegador nunca envía el monto.
notification-service	security-service GET /internal/v1/users/{id}/contact (encabezado X-Internal-Key)	Correo vigente, nombre, estado activo e interruptores de notificación del usuario (ADR-011, secciones 8 y 12).
notification-service	security-service GET /internal/v1/users/admins (encabezado X-Internal-Key)	Administradores activos que reciben los avisos al personal (ADR-011, sección 10).
booking-service	payment-service GET /internal/v1/payments/bookings/{bookingId}/reported (encabezado X-Internal-Key, 3 s; *PAYMENT_SERVICE_URL*)	Desde el 26 de octubre de 2026: antes de cancelar por falta de pago, saber si la reserva ya tiene un pago en revisión o aprobado (*PaymentStatusDirectory*, *PaymentServiceDirectory*).
api-gateway	security-service GET /api/v1/auth/session (el mismo token)	Confirmar que la sesión del token siga abierta antes de pasar la petición (desde el 21 de octubre de 2026; sección 9.2).

Nota. Si el destino no responde, quien llama responde 503 con un código estable (CUSTOMER_SERVICE_UNAVAILABLE o BOOKING_SERVICE_UNAVAILABLE). Tiempos de espera por defecto: 3 s (5 s en payment-service).

9.4.2 Comunicación Asíncrona con RabbitMQ

RabbitMQ se usa para el trabajo que puede ocurrir después de la transacción original, como crear el perfil del cliente y las notificaciones. Los eventos se publican después del commit, solo cuando *MESSAGING_ENABLED=true*; en otro caso se escriben en el registro. El catálogo de eventos está en la sección 13.4.

9.5 Aspectos Transversales

9.5.1 Bloque de Auditoría y Ciclo de Vida de los Datos

Toda tabla de dominio lleva siete columnas de auditoría: *created_at*, *created_by*, *updated_at*, *updated_by*, *deleted_at*, *deleted_by* y *row_version*. Los datos de dominio usan borrado lógico (*deleted_at*). Un disparador `tr_touch_<tabla>` actualiza *updated_at* e incrementa *row_version* en cada UPDATE, por lo que el código nunca los escribe.

El diseño también prevé la tabla *audit.audit_log*, alimentada por disparadores que leen el autor de *SESSION_CONTEXT(N'user_id')*. **En la versión actual la tabla existe, pero nadie escribe en ella**: los changelogs no crean disparadores de auditoría y los servicios no fijan el contexto de sesión. Se registra como discrepancia (Anexo B) y como acción ACO-04 del documento de acciones correctivas, preventivas y de mejoramiento.

9.5.2 Concurrencia y Transacciones

La columna *row_version* existe, pero las entidades todavía no se mapean con *@Version* (TD-008), por lo que en las ediciones administrativas concurrentes gana la última escritura. Las superposiciones críticas las protege la base de datos: el disparador *tr_booking_bay_overlap* impide que dos reservas activas ocupen la misma bahía en horarios superpuestos, *tr_operator_absence_overlap* impide ausencias superpuestas y la columna *sequence_no*, única por cliente, impide que dos movimientos del libro de puntos se calculen sobre el mismo saldo. Donde dos operaciones a la vez podrían chocar, los servicios Java toman un candado de aplicación de SQL Server (*sp_getapplock*, dueño *Transaction*) que se suelta al terminar la transacción: booking-service usa *booking-schedule* al crear o reprogramar una reserva y al cerrar las vencidas, y operations-service usa *operations-assignment* en la asignación manual y la automática (desde el 21 de octubre de 2026), con una espera máxima de 10 segundos. Cada caso de uso abre una transacción local que solo escribe los esquemas de su propio servicio; no hay transacciones distribuidas.

9.5.3 Registros, Correlación y Errores

Los servicios escriben sus registros en la salida estándar con el nivel, el nombre del servicio y el identificador de correlación (*CorrelationIdFilter* en los servicios Java y *CorrelationIdMiddleware* en payment-service, clave *correlationId*, encabezado *X-Correlation-Id*). El encabezado se reenvía en las llamadas de booking-service a customer-service, de operations-service a booking-service y security-service, y de payment-service a booking-service. Falta reenviarlo en las llamadas de customer-service y notification-service a security-service y en los mensajes de RabbitMQ, y que el gateway lo genere (TD-007). Las contraseñas, los tokens, los códigos de recuperación y el contenido de los comprobantes nunca se registran.

Los errores siguen el RFC 9457, *Problem Details for HTTP APIs* (Nottingham et al., 2023): el cuerpo es *application/problem+json* con un *code* estable que la aplicación web traduce con la clave `API_ERRORS.<code>`. Las respuestas no incluyen trazas de pila ni mensajes internos (*server.error.include-message: never*). La Tabla 8 resume los códigos de estado.

Tabla 8

Códigos de Estado HTTP que Usan los Servicios

Código	Significado	Ejemplos de code
400	Solicitud o entrada no válida.	VALIDATION_ERROR, INVALID_PLATE, UNKNOWN_PERMISSION, NOTIFICATION_TYPE_NOT_ALLOWED, RECEIPT_TOO_LARGE
401	Autenticación ausente, no válida o vencida, o sesión cerrada.	UNAUTHORIZED, INVALID_CREDENTIALS
403	Autenticado, pero no autorizado.	FORBIDDEN, ACCOUNT_DISABLED, CANNOT_DISABLE_OWN_ACCOUNT, ADMIN_CANNOT_BE_DISABLED
404	Recurso no encontrado o de otro usuario.	VEHICLE_NOT_FOUND, BOOKING_NOT_FOUND
409	Conflicto con el estado actual.	EMAIL_ALREADY_REGISTERED, SLOT_UNAVAILABLE, OPERATOR_BUSY, PAYMENT_ALREADY_REPORTED, TRANSACTION_REFERENCE_REUSED, PROMOTION_ALREADY_REDEEMED
503	Dependencia no disponible.	CUSTOMER_SERVICE_UNAVAILABLE, BOOKING_SERVICE_UNAVAILABLE

Nota. Códigos tomados de los manejadores de excepciones de los servicios y del archivo de traducción de la aplicación web.

9.5.4 Tiempo e Idioma

Todas las marcas de tiempo se guardan en UTC (*DATETIME2(3)*, *SYSUTCDATETIME()*). Las fechas y horas de las reservas se intercambian como fechas locales del lavadero (*yyyy-MM-dd* y *HH:mm*) en la zona *BOOKING_ZONE* (America/Bogota), y las respuestas también llevan los instantes en UTC. El backend devuelve códigos estables, y los clientes los traducen al español, inglés, francés o portugués.

9.5.5 Patrones No Adoptados

El proyecto decidió no adoptar en esta versión Outbox, Saga, CQRS, Event Sourcing, BFF, malla de servicios, Redis, base de datos por servicio, interruptor de circuito formal ni pasarela de pagos. La consecuencia más importante es la ausencia de Outbox: un evento se puede perder si el servicio se detiene entre el commit y la publicación (TD-011).

10. Estructura del Proyecto

10.1 Repositorios

El proyecto se distribuye en once repositorios de Git, clonados uno al lado del otro (Tabla 9).

Tabla 9

Repositorios del Proyecto

Repositorio	Contenido
lavarapido-infra	docker-compose.yml, .env.example, README y guía del equipo (GUIA-EQUIPO.md).
lavarapido-security-service, lavarapido-customer-service, lavarapido-booking-service, lavarapido-operation-service (en singular; servicio *operations-service*), lavarapido-notification-service	Servicios Java (Maven Wrapper, Dockerfile, *src/main*, *src/test*).
lavarapido-payment-service	Solución .NET 8, Dockerfile, changelogs de Liquibase en *db/* y su propio docker-compose para probar las migraciones.
lavarapido-api-gateway	Spring Cloud Gateway.
Front-end-proyecto-web	Aplicación Angular.
LavadoVehicular-Movil	Aplicación React Native con Expo.
vehicle-w-docs	Documentación: gobierno, requisitos, arquitectura y ADR, datos, contratos de la API, UML, servicios, DevOps, calidad, UX, operación y control del proyecto.

Nota. Las direcciones de los repositorios se listan en el Manual de Instalación, sección 8.

10.2 Estructura de un Servicio Java

El siguiente árbol corresponde a security-service; los demás servicios Java siguen la misma organización (`com.lavarapido.<servicio>`).

```
lavarapido-security-service/
|-- Dockerfile                     (construcción con Temurin 21 JDK, ejecución con Temurin 21 JRE)
|-- pom.xml, mvnw, mvnw.cmd
`-- src/
    |-- main/java/com/lavarapido/security/
    |   |-- SecurityServiceApplication.java
    |   |-- domain/
    |   |   |-- model/        UserAccount, Person, EmailAddress, HashedPassword, RoleCode, UserPreference...
    |   |   |-- service/      PasswordPolicy, PasswordRecoveryPolicy
    |   |   |-- event/        UserRegistered, PasswordChanged, EmailChanged, AccountDeactivated
    |   |   |-- exception/    InvalidCredentialsException, WeakPasswordException...
    |   |   `-- port/in, port/out
    |   |-- application/usecase/   AuthenticationService, AccountRegistrationService,
    |   |                          PasswordRecoveryService, UserAdministrationService...
    |   `-- infrastructure/
    |       |-- adapter/in/web/          AuthController, UserProfileController,
    |       |                            UserAdministrationController, InternalUserController, dto/
    |       |-- adapter/out/persistence/ entity/, repository/, *PersistenceAdapter
    |       |-- adapter/out/messaging/   RabbitDomainEventPublisher, LoggingDomainEventPublisher
    |       |-- adapter/out/notification/ EmailResetCodeNotifier, LoggingResetCodeNotifier
    |       |-- adapter/out/security/    BCryptPasswordHasher, JwtTokenIssuer...
    |       `-- config/                  SecurityConfig, JwtConfig, CorrelationIdFilter,
    |                                    InternalApiKeyFilter, DevDataSeeder, OpenApiConfig
    |-- main/resources/
    |   |-- application.yml, application-dev.yml
    |   `-- db/changelog/  db.changelog-master.xml, 001-..., 002-..., 003-..., 020-..., 023-..., 026-..., 030-...,
seed/101-...
    `-- test/java/...      pruebas de dominio, casos de uso, seguridad y web
```

10.3 Estructura de payment-service

```
lavarapido-payment-service/
|-- PaymentService.sln, Directory.Build.props (net8.0), Dockerfile
|-- db/changelog/db.changelog-master.yaml, changes/001 ... 020
|-- src/
|   |-- PaymentService.Domain/          Payments/, Promotions/, Loyalty/, Common/
|   |-- PaymentService.Application/     Payments/ (PaymentCommandService, PaymentQueryService,
|   |                                   BookingPaymentGuard, AmountDueCalculator), PaymentAccounts/,
|   |                                   Promotions/, Loyalty/ (LoyaltyService, LoyaltyRewardService),
|   |                                   Ports/In, Ports/Out, Common/
|   |-- PaymentService.Infrastructure/  Persistence/, Booking/BookingServiceDirectory.cs, Messaging/
|   `-- PaymentService.Api/             Controllers/ (Payments, AdminPayments, AdminPaymentAccounts,
|                                       Promotions, Loyalty), Program.cs, Errors/, Configuration/
`-- tests/  PaymentService.Domain.UnitTests, PaymentService.Application.UnitTests,
            PaymentService.Infrastructure.IntegrationTests
```

10.4 Estructura de la Aplicación Web

```
Front-end-proyecto-web/src/
|-- app/
|   |-- app.routes.ts            (/, /auth, /client, /operator, /admin, /report/:token)
|   |-- core/
|   |   |-- constants/  api.ts (API_BASE_URL = '/api/v1'), payment-texts.ts (temporal)
|   |   |-- guards/     auth-guard.ts (sesión y rol por área)
|   |   |-- interceptors/ auth-interceptor.ts (token Bearer; sesión vencida -> /auth)
|   |   |-- models/, utils/ (api-error.ts traduce los códigos de error; image-compress.ts deja el comprobante en máximo 2
MB)
|   |   `-- services/   auth (sesión sincronizada entre pestañas), booking-api, operations-api,
|   |                   payments-api, promotions-api, loyalty-api, role-permissions-api,
|   |                   notifications, vehicles, user-admin, preferences, inspection-api,
|   |                   inspection-photos...
|   |-- features/
|   |   |-- landing/        página de bienvenida
|   |   |-- auth/           login, register, forgot-password (3 pasos)
|   |   |-- client/         dashboard, vehicles, reserve, payment, history, notifications, profile, configuration
|   |   |-- operator/       home, schedule, assigned-services, service-history, qualifications, notifications...
|   |   |-- admin/          dashboard, reservations, inspection, payments, management, schedule,
|   |   |                   operators, operator-detail, operator-calendar, reports, settings...
|   |   `-- public-report/  informe público de inspección
|   |-- layout/, shared/    sidebar, dialogs (modales), components, pipes
|-- assets/i18n/            es.json, en.json, fr.json, pt.json
`-- proxy.conf.json         /api -> http://localhost:8080
```

La aplicación no usa zonas (*zoneless*) y trabaja con señales; cada área tiene un módulo de carga diferida protegido por *authGuard* con el rol requerido (CLIENT, OPERATOR o ADMIN), y las páginas de autenticación usan *guestGuard*.

Patrón MVVM. Cada página de *features/\<área\>/pages* es la vista: un componente *standalone* que solo muestra y reenvía eventos. Su lógica está en un ViewModel (*@Injectable*) de *features/\<área\>/viewmodels* o, si la comparten varias áreas (perfil, configuración, crear usuario, barra lateral), de *shared/viewmodels*: guarda el estado en señales (*signal*, *computed*), valida los formularios y llama a los servicios de *core/services*, que son el modelo. La página lo recibe por *providers* e *inject()*. Al 23 de octubre de 2026 hay 36 ViewModels (14 del administrador, 6 del operario, 5 del cliente, 3 de autenticación, 1 de la bienvenida, 1 del informe público y 6 compartidos); se pueden probar con Vitest sin montar la vista y 9 tienen ya su **.viewmodel.spec.ts*. Algunos ViewModels del administrador exponen *getters* en lugar de señales (por ejemplo *reservations.viewmodel.ts*). Una regla nueva de una pantalla va en su ViewModel, no en la plantilla ni en el componente.

10.5 Estructura de la Aplicación Móvil

La aplicación móvil se organiza por capas y por rol. Todas las llamadas van al gateway mediante *EXPO_PUBLIC_API_URL*:

```
LavadoVehicular-Movil/src/
|-- app/        config (i18n y locales es, en, fr, pt), providers (sesión, tema, AppUpdatesCheck,
|               sincronización de push), theme
|-- core/       api (cliente fetch y apiError, que traduce los códigos de error), services (un cliente
|               por servicio del backend), navigation (RootNavigator: cada rol solo registra sus
|               pantallas), storage (sesión en almacenamiento seguro, SecureStore)
|-- modules/    auth, client, operator, admin: cada uno con screens (vista), viewmodels (hooks use*.ts
|               con el estado y las acciones), components, hooks, models y constants
`-- shared/     components, pantallas compartidas (notificaciones, perfil, configuración), layouts
                (barra inferior y menú "Más" por rol), validators, utils, hooks, i18n
```

La app sigue la misma idea MVVM que la web con hooks de React: la pantalla de *screens* dibuja y llama a su hook de *viewmodels* (por ejemplo *useLoginViewModel*, *useInspection* o *useSettings*; 20 en total), que guarda el estado y usa los servicios de *core/services*. Hay 31 pantallas: 3 de autenticación, 8 del cliente, 8 del operario y 12 del administrador. Todavía hay 10 pantallas o componentes que llaman a los servicios sin pasar por un ViewModel (por ejemplo *ClientPaymentScreen*, *ClientReserveScreen* y *OperatorHomeScreen*); está registrado como deuda técnica. Las notificaciones push siguen el patrón Command (*RegisterDeviceCommand* y *UnregisterDeviceCommand*). Desde el 23 de octubre de 2026 la app tiene sus primeras pruebas con *jest-expo* (*src/shared/validators/authValidators.test.ts*, *src/modules/admin/utils/managementUtils.test.ts* y *src/core/services/__tests__/services.test.ts*, 13 pruebas), que el CI corre con *npm test -- --ci*.

El informe de inspección del administrador (RF-036) sigue la misma organización que el resto del módulo: *core/services/operations/InspectionService.ts* llama a los mismos endpoints que la web (*/admin/inspections/...*); *modules/admin/viewmodels/useInspection.ts* guarda el estado que devuelve el servidor; *modules/admin/services/inspectionPhotos.ts* toma o elige las fotos con *expo-image-picker* y las reduce con *expo-image-manipulator* (lado mayor de 1600 px, JPEG), porque el servidor acepta como máximo 2 MB por foto; *modules/admin/components/inspection* contiene la tarjeta de cada fase, el formulario de hallazgo y la foto ampliada; y *modules/admin/screens/AdminInspectionScreen.tsx* es la pantalla, que se abre desde el detalle de la reserva (ruta de navegación *AdminInspection*). Los textos de este módulo están en *modules/admin/constants/inspectionTexts.ts*, en los cuatro idiomas.

Desde el 21 de octubre de 2026, *core/services/payments/receiptImage.ts* comprime el comprobante de pago a máximo 2 MB antes de subirlo, *shared/components/feedback/ImageZoomModal.tsx* amplía el QR de la cuenta de pago y *app/providers/AppUpdatesCheck.tsx* lee */app/version.json* al abrir el APK instalado: si hay un *versionCode* mayor, muestra encima de todo una pantalla que no se cierra y solo deja descargar el APK nuevo; si el servidor no responde, la aplicación sigue (*core/services/appUpdates/publishedApk.ts*).

Actualizaciones (APK y EAS Update). La app recibe los cambios por dos vías, que decide *app/providers/AppUpdatesCheck.tsx*. Si el servidor publica un APK con *versionCode* mayor (*/app/version.json*), la app se bloquea hasta descargarlo: es la única forma de entregar cambios nativos (un paquete nativo, permisos, ícono o *app.json*); se compila con *eas build -p android --profile preview* y se publica con *npm run publish:apk*. Los cambios solo de pantallas y lógica llegan por internet con EAS Update (*expo-updates*, canal *preview*, *runtimeVersion* con política *fingerprint* y revisión al abrir la app, *checkAutomatically: ON_LOAD*): la app baja la actualización y ofrece reiniciar; se publican con *npm run publish:update -- "qué cambió"* (*scripts/publish-update.mjs*, que fija la misma dirección del backend que el APK). Una actualización por internet no llega a un APK con otra *runtimeVersion*. La versión de la app es 1.1.0 (*versionCode* 5).

11. Base de Datos

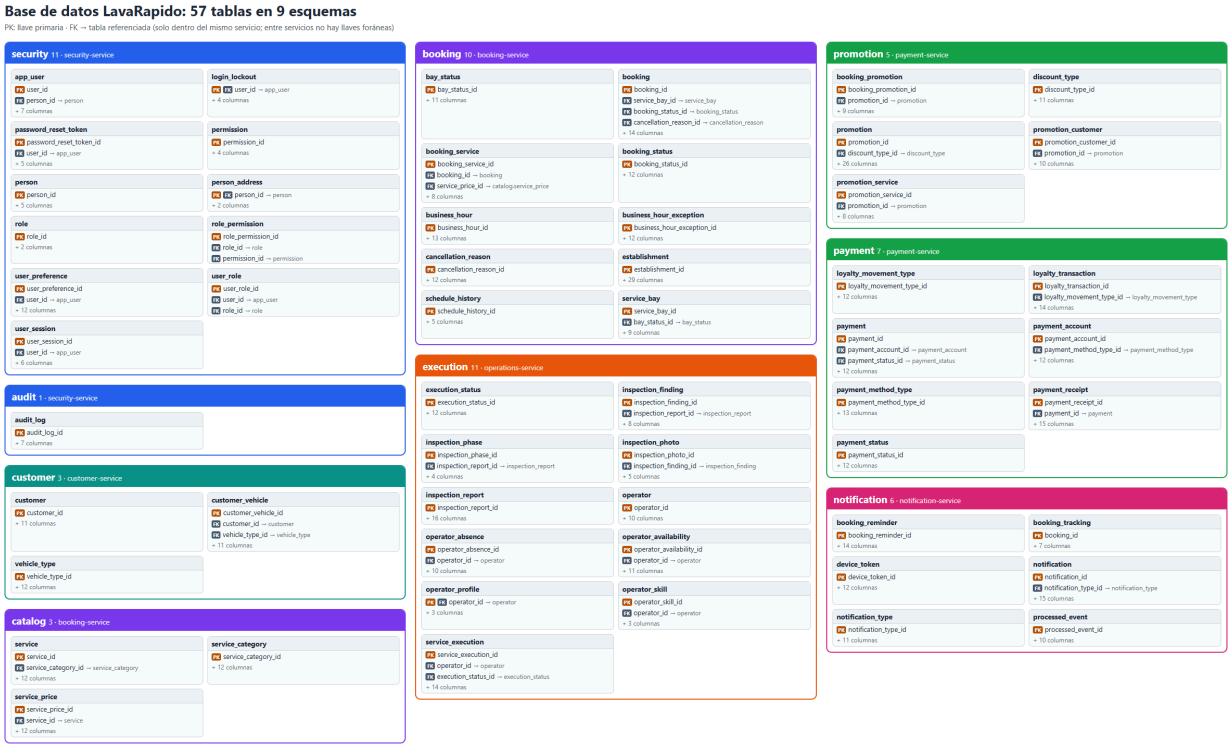
El sistema usa una sola base de datos SQL Server 2022 llamada *LavaRapido*, dividida en nueve esquemas que pertenecen a seis servicios, con 57 tablas.

11.1 Diagrama Entidad-Relación

La Figura 2 se generó directamente desde la base de datos *LavaRapido* (catálogo *sys.tables*, *sys.columns* y *sys.foreign_keys*) con todos los servicios arrancados. Muestra cada tabla dentro de su esquema, sus llaves primarias y sus llaves foráneas con la tabla a la que apuntan; las demás columnas se resumen como un número. Las referencias entre servicios (por ejemplo, *execution.operator.user_id* hacia *security.app_user*) no aparecen porque no tienen llave foránea (Tabla 10).

Figura 2

Diagrama Entidad-Relación del Sistema



11.2 Tablas Principales

11.2.1 Propiedad de los Esquemas y Reglas de Claves Foráneas

Las claves foráneas solo se permiten dentro del límite de un servicio. Entre servicios, las referencias son columnas indexadas cuyo valor valida la aplicación antes de escribir (ADR-003). Por ejemplo, *booking.booking_service.service_price_id* es una clave foránea hacia *catalog.service_price* porque ambos esquemas pertenecen a booking-service, mientras que *payment.payment.booking_id* no lo es (Tabla 10).

Tabla 10

Referencias entre Servicios sin Clave Foránea

Columna	Apunta a	Dueño del destino
customer.customer.person_id, customer.customer.user_id	security.person, security.app_user	security-service
catalog.service_price.vehicle_type_id	customer.vehicle_type	customer-service
booking.booking.customer_vehicle_id	customer.customer_vehicle	customer-service
booking.booking.booked_by	security.app_user	security-service
execution.operator.user_id	security.app_user	security-service
execution.service_execution.booking_service_id	booking.booking_service	booking-service
promotion.promotion_service.service_id	catalog.service	booking-service
promotion.promotion_customer.customer_id, promotion.booking_promotion.booking_id	customer.customer, booking.booking	customer-service, booking-service
payment.payment.booking_id, payment.loyalty_transaction.booking_id	booking.booking	booking-service
payment.payment.approved_by, payment.payment_receipt.uploaded_by, reviewed_by	security.app_user	security-service
payment.loyalty_transaction.customer_id	security.app_user (guarda el user_id del cliente, ADR-015)	security-service
notification.notification.user_id y los user_id de device_token, booking_reminder y booking_tracking	security.app_user	security-service

Nota. Elaborado por los autores a partir de los changelogs y de los modelos de datos de los servicios.

11.2.2 Convenciones de Modelado

- **Claves sustitutas:** enteros IDENTITY(1,1) (BIGINT en tablas de alto crecimiento, INT o SMALLINT en catálogos), no UUID.
- **Bloque de auditoría:** siete columnas en toda tabla de dominio, salvo el núcleo compartido de seguridad.
- **Borrado lógico:** toda consulta debe incluir *deleted_at IS NULL*; las reglas de unicidad del negocio usan índices filtrados con esa condición (por ejemplo, la placa es única solo entre los vehículos activos).
- **Tiempo:** DATETIME2(3) en UTC.
- **Nombres:** esquemas de una sola palabra; tablas en snake_case singular; claves primarias `<tabla>_id`; restricciones *pk_*, *fk_*, *uq_*, *ck_*; índices *ix_* y *ux_*; disparadores *tr_* y *tr_touch_*. *[role]*, *[permission]* y *[action]* van entre corchetes porque son palabras reservadas.
- **Catálogos:** tablas separadas con la forma *code*, *name*, *display_order* e *is_active*, para que una clave foránea impida mezclar estados de entidades distintas.
- **Códigos de estado:** en inglés en la base de datos (SCHEDULED, CONFIRMED, IN_PROGRESS, COMPLETED, CANCELLED, NO_SHOW; PENDING, IN_REVIEW, APPROVED, REJECTED, REFUNDED; PENDING, IN_PROGRESS, PAUSED, COMPLETED, WITH_ISSUE). Los códigos de los métodos de pago siguen en español (EFECTIVO, NEQUI, DAVIPLATA, TRANSFERENCIA).

11.2.3 Tablas por Esquema

Tabla 11

Tablas del Sistema por Esquema

Esquema (dueño)	Tablas
security (security-service)	person, person_address, app_user, role, permission, role_permission, user_role, user_session, password_reset_token, user_preference, login_lockout
audit (security-service)	audit_log
customer (customer-service)	vehicle_type, customer, customer_vehicle
catalog (booking-service)	service_category, service, service_price
booking (booking-service)	establishment, business_hour, business_hour_exception, bay_status, service_bay, booking_status, cancellation_reason, booking, booking_service, schedule_history
execution (operations-service)	execution_status, operator, operator_availability, operator_absence, service_execution, operator_profile, operator_skill, inspection_report, inspection_phase, inspection_finding, inspection_photo
promotion (payment-service)	discount_type, promotion, promotion_service, promotion_customer, booking_promotion
payment (payment-service)	payment_method_type, payment_account, payment_status, payment, payment_receipt, loyalty_movement_type, loyalty_transaction
notification (notification-service)	notification_type, notification, device_token, processed_event, booking_reminder, booking_tracking

Nota. Elaborado por los autores a partir de la estrategia de migraciones y de los modelos de datos de los servicios.

11.3 Diccionario de Datos

El diccionario lista, para cada tabla, sus columnas, tipos, nulabilidad y restricciones. Para que sea legible, el bloque de auditoría estándar (siete columnas) se resume como "bloque de auditoría".

Tabla 12

Diccionario de Datos de los Esquemas de Seguridad y Auditoría

Tabla	Campo	Tipo	Nulo	Restricción o descripción
security.person	person_id	BIGINT	No	PK
	document_number	NVARCHAR(20)	No	Único (uq_person_document)
	first_name, last_name	NVARCHAR(60)	No	—
	phone	NVARCHAR(20)	Sí	—
	email	NVARCHAR(120)	Sí	—
security.app_user	user_id	BIGINT	No	PK
	person_id	BIGINT	No	FK a person; único
	username	NVARCHAR(60)	No	Único; es el correo
	password_hash	NVARCHAR(255)	No	Hash BCrypt
	is_active	BIT	No	Por defecto 1
	last_login	DATETIME2(3)	Sí	—
	terms_accepted_at, data_policy_accepted_at	DATETIME2(3)	Sí	Fecha de aceptación de los Términos y de la Política de datos al registrarse (changelog 039); nulo en cuentas anteriores
	created_at	DATETIME2(3)	No	Fecha de creación de la cuenta (changelog 036)

Tabla	Campo	Tipo	Nulo	Restricción o descripción
security.role	role_id, code, name	SMALLINT, NVARCHAR(30), NVARCHAR(60)	No	PK; code único (ADMIN, OPERATOR, CLIENT)
security.permission	permission_id, code, name, resource, action	SMALLINT, NVARCHAR	No	PK; code único; catálogo cargado por el changelog 026
security.role_permission	role_permission_id, role_id, permission_id	BIGINT, SMALLINT, SMALLINT	No	FK; único (role_id, permission_id); matriz informativa que edita el administrador (ADR-015)
security.user_role	user_role_id, user_id, role_id	BIGINT, BIGINT, SMALLINT	No	FK; único (user_id, role_id)
security.user_session	user_session_id, user_id	BIGINT	No	PK; FK a app_user
	refresh_token_hash	NVARCHAR(255)	Sí	No se usa (no hay tokens de actualización)
	started_at, expires_at	DATETIME2(3)	No	—
	revoked_at	DATETIME2(3)	Sí	Se fija al cerrar sesión
	ip_address, user_agent	NVARCHAR(45), NVARCHAR(300)	Sí	—
security.password_reset_token	password_reset_token_id, user_id	BIGINT	No	PK; FK a app_user
	token_hash	NVARCHAR(255)	No	Único; hash BCrypt del código
	requested_at, expires_at	DATETIME2(3)	No	ck: expires_at > requested_at
	used_at, ip_address	DATETIME2(3), NVARCHAR(45)	Sí	—
security.user_preference	user_preference_id, user_id	BIGINT	No	PK; FK y user_id único
	theme	NVARCHAR(20)	No	green-light, green-dark, pink o pink-dark
	language	NVARCHAR(5)	No	es, en, fr o pt
	notifications_enabled	BIT	No	Por defecto 1; interruptor de push
	email_reminders_enabled, promotions_enabled	BIT	No	Por defecto 1 (changelog 030); bloque de auditoría
security.person_address	person_id, address	BIGINT, NVARCHAR(200)	No	PK y FK a person; dirección de contacto del perfil (changelog 035)
security.login_lockout	user_id, failed_count, first_failed_at, locked_until	BIGINT, INT, DATETIME2(3)	No/Sí	PK y FK a app_user; intentos fallidos y bloqueo temporal (changelog 034)
audit.audit_log	audit_log_id	BIGINT	No	PK
	entity, entity_id, action	NVARCHAR(60), BIGINT, NVARCHAR(20)	No	—
	previous_data, new_data	NVARCHAR(MAX)	Sí	Texto JSON
	changed_by	BIGINT	Sí	—
	changed_at	DATETIME2(3)	No	Por defecto SYSUTCDATETIME()

Nota. Fuente: changelogs 001, 002, 003, 020, 026, 030, 034, 035, 036 y 039 de security-service. El changelog 023 creó *security.custom_role* y el 026 la eliminó (ADR-015).

Tabla 13

Diccionario de Datos del Esquema de Clientes

Tabla	Campo	Tipo	Nulo	Restricción o descripción
customer.vehicle_type	vehicle_type_id	SMALLINT	No	PK; identificadores 1 a 6 fijos
	code, name	NVARCHAR(30), NVARCHAR(60)	No	CAR, SEDAN, SUV, PICKUP, TRUCK, MOTO
	size_factor	DECIMAL(4,2)	No	> 0 (0,80 a 1,35); sugerencia de precio
	display_order, is_active	SMALLINT, BIT	No	Bloque de auditoría
customer.customer	customer_id	BIGINT	No	PK
	person_id	BIGINT	No	Único; referencia a security.person
	user_id	BIGINT	Sí	Único entre las filas activas; el *sub* del JWT
	loyalty_points	INT	No	≥ 0; copia de presentación; el saldo oficial está en payment.loyalty_transaction (ADR-015)
	customer_since	DATE	No	Bloque de auditoría
customer.customer_vehicle	customer_vehicle_id, customer_id	BIGINT	No	PK; FK a customer
	license_plate	NVARCHAR(10)	No	Única entre los vehículos activos (ux_customer_vehicle_plate_active)
	vehicle_type_id	SMALLINT	No	FK a vehicle_type
	brand, model, color	NVARCHAR(50), NVARCHAR(50), NVARCHAR(30)	Sí	Bloque de auditoría

Nota. Fuente: changelogs 004, 014, 015 y semilla 102 de customer-service.

Tabla 14

Diccionario de Datos de los Esquemas de Catálogo y Reservas

Tabla	Campo	Tipo	Nulo	Restricción o descripción
catalog.service_category	service_category_id, code, name, description, display_order, is_active	SMALLINT, NVARCHAR	No/Sí	LAVADO, POLICHADO, DETALLADO, INTERIOR, PAQUETES
catalog.service	service_id, code, name, description, service_category_id, is_active	INT, NVARCHAR	No/Sí	FK a service_category; BASIC, PREMIUM, FULL
	loyalty_points	INT	No	Por defecto 0 (changelog 022); puntos que gana el cliente cuando se aprueba el pago
catalog.service_price	service_price_id, service_id, vehicle_type_id	BIGINT, INT, SMALLINT	No	FK a service; un precio vigente por par (ux_service_price_current)
	price, estimated_minutes	DECIMAL(12,2), SMALLINT	No	Inmutables (tr_service_price_immutable)
	valid_from, valid_to	DATE	No/Sí	Bloque de auditoría
booking.establishment	establishment_id, legal_name, trade_name, tax_id, address, phone, email, logo_url, singleton; datos del negocio (changelog 032): business_type, founded_on, legal_representative, legal_representative_document, website, tax_regime, ciiu_activity, dian_resolution, invoice_prefix, invoice_range, instagram, facebook, support_line, service_hours_note	SMALLINT, NVARCHAR, DATE, BIT	No/Sí	Una sola fila (singleton único); la editan GET y PUT /api/v1/admin/establishment

Tabla	Campo	Tipo	Nulo	Restricción o descripción
booking.business_hour	business_hour_id, day_of_week, opens_at, closes_at, break_starts_at, break_ends_at, is_active	SMALLINT, TIME(0), BIT	No/Sí	Una fila por día (1 = lunes)
booking.business_hour_exception	business_hour_exception_id, exception_date, is_closed, opens_at, closes_at, reason	INT, DATE, BIT, TIME(0), NVARCHAR(120)	No/Sí	Fecha única
booking.bay_status	bay_status_id, code, name, accepts_bookings, display_order	SMALLINT, NVARCHAR, BIT	No	ACTIVE, MAINTENANCE, INACTIVE
booking.service_bay	service_bay_id, code, name, bay_status_id	SMALLINT, NVARCHAR, SMALLINT	No	FK a bay_status; cuatro bahías en la semilla
booking.booking_status	booking_status_id, code, name, is_final, display_order, is_active	SMALLINT, NVARCHAR, BIT	No	Identificadores 1 a 6; 5 = CANCELLED
booking.cancellation_reason	cancellation_reason_id, code, name, is_customer_fault, display_order, is_active	SMALLINT, NVARCHAR, BIT	No	—
booking.booking	booking_id, customer_vehicle_id, service_bay_id, booking_status_id, booked_by	BIGINT, SMALLINT	No/Sí	FK a service_bay y booking_status; disparador de superposición
	scheduled_start, scheduled_end	DATETIME2(3)	No	UTC
	cancellation_reason_id	SMALLINT	Sí	Solo con el estado 5 (ck_booking_cancel)
	points_redeemed, points_discount_amount, notes	INT, DECIMAL(12,2), NVARCHAR(300)	No/Sí	Mecanismo de puntos distinto y sin uso (quedan en 0); no hay columna de total
booking.booking_service	booking_service_id, booking_id, service_price_id, quantity	BIGINT, SMALLINT	No	FK a booking y a catalog.service_price
booking.schedule_history	schedule_history_id, entity_type, title, detail, changed_at, changed_by	BIGINT, NVARCHAR, DATETIME2(3)	No/Sí	Registro de los cambios del horario, las excepciones y las bahías (changelog 021)

Nota. Fuente: changelogs 005, 018, 021, 022 y 024 de booking-service. El changelog 024 retiró *service_bay.operator_id* (ADR-015).

Cada tabla tiene también el bloque de auditoría, salvo el registro de historial.

Tabla 15

Diccionario de Datos del Esquema de Ejecución

Tabla	Campo	Tipo	Nulo	Restricción o descripción
execution.execution_status	execution_status_id, code, name, is_final, display_order, is_active	SMALLINT, NVARCHAR, BIT	No	PENDING, IN_PROGRESS, PAUSED, COMPLETED, WITH_ISSUE
execution.operator	operator_id, user_id, hired_on, is_active	INT, BIGINT, DATE, BIT	No	user_id único; se crea automáticamente para las cuentas OPERATOR
execution.operator_availability	operator_availability_id, operator_id, day_of_week, starts_at, ends_at, is_active	INT, SMALLINT, TIME(0), BIT	No	Una fila por operario y día; starts_at < ends_at
execution.operator_absence	operator_absence_id, operator_id, starts_at, ends_at, reason	INT, DATETIME2(3), NVARCHAR(120)	No	Sin superposición (tr_operator_absence_overlap)
execution.service_execution	service_execution_id, booking_service_id, operator_id, execution_status_id	BIGINT, INT, SMALLINT	No	booking_service_id único
	started_at, finished_at	DATETIME2(3)	Sí	—
	follow_up_notice_at	DATETIME2(3)	Sí	Cuándo se avisó que el servicio terminó sin registrarse; se borra al iniciarlo (changelog 040)

Tabla	Campo	Tipo	Nulo	Restricción o descripción
	quality_rating, quality_comment, rated_at	SMALLINT, NVARCHAR(500), DATETIME2(3)	Sí	Calificación de 1 a 5; comentario y fecha solo con calificación
	is_comment_visible	BIT	No	Bloque de auditoría
execution.operator_profile	operator_id, specialty	INT, NVARCHAR(80)	No/Sí	PK y FK a operator; especialidad del operario (changelog 037)
execution.operator_skill	operator_skill_id, operator_id, name, skill_level, display_order	INT, NVARCHAR(60), NVARCHAR(12), SMALLINT	No	FK a operator; nombre único por operario; nivel CERTIFIED, EXPERT o ADVANCED (ck_operator_skill_level)
execution.inspection_report	inspection_report_id, booking_id, booking_code, vehicle, plate, services, booking_date, booking_time, duration_minutes, public_token, is_published, published_at	BIGINT, NVARCHAR, DATE, TIME, INT, BIT, DATETIME2(3)	No/Sí	Un informe por reserva (uq_inspection_report_booking); token público único (changelog 033)
execution.inspection_phase	inspection_phase_id, inspection_report_id, phase, status, started_at, finished_at	BIGINT, NVARCHAR(20), DATETIME2(3)	No/Sí	FK a inspection_report; una fila por fase (uq_inspection_phase); fase y estado validados por restricciones ck
execution.inspection_finding	inspection_finding_id, inspection_report_id, phase, area, severity, note	BIGINT, NVARCHAR	No/Sí	FK a inspection_report; severidad INFO, MINOR o MAJOR (ck_inspfinding_severity)
execution.inspection_photo	inspection_photo_id, inspection_finding_id, content_type, content, size_bytes	BIGINT, NVARCHAR(30), VARBINARY(MAX), INT	No	FK a inspection_finding; JPG o PNG de hasta 2 MB

Nota. Fuente: changelogs 007, 033, 037 y 040 y semilla 105 de operations-service.

Tabla 16

Diccionario de Datos de los Esquemas de Promociones y Pagos

Tabla	Campo	Tipo	Nulo	Restricción o descripción
promotion.discount_type	discount_type_id, code, name, display_order, is_active	SMALLINT, NVARCHAR	No	PERCENT, FIXED
promotion.promotion	promotion_id, code, name, description, discount_type_id, discount_value, max_discount_amount, min_purchase_amount, is_public, min_completed_booking, valid_from, valid_to, max_redemptions, max_redemptions_per_customer, is_active	INT, NVARCHAR, DECIMAL, BIT, DATE	No/Sí	discount_type PERCENT con discount_value de 1 a 100 (ADR-015)
	price, duration_minutes, icon, featured, benefits	DECIMAL, SMALLINT, NVARCHAR, BIT	Sí	Campos de presentación (changeset 017); precio y duración opcionales
	required_points	INT	No	Puntos necesarios para desbloquear el cupón; 0 = sin requisito (changeset 018)
promotion.promotion_service, promotion_customer, booking_promotion	Vínculos de una promoción con servicios, clientes y reservas	INT, BIGINT, DECIMAL	No/Sí	booking_promotion registra cada canje (uno por cupón y reserva)

Tabla	Campo	Tipo	Nulo	Restricción o descripción
payment.payment_method_type	payment_method_type_id, code, name, requires_account, requires_receipt, display_order, is_active	SMALLINT, NVARCHAR, BIT	No	EFECTIVO, NEQUI, DAVIPLATA, TRANSFERENCIA
payment.payment_account	payment_account_id, payment_method_type_id	SMALLINT	No	PK; FK a payment_method_type
	account_holder, account_number, instructions	NVARCHAR(120), NVARCHAR(50), NVARCHAR(300)	No/Sí	—
	qr_image_url	NVARCHAR(MAX)	Sí	Imagen del QR como URL de datos (changeset 016)
	is_active	BIT	No	Bloque de auditoría
payment.payment_status	payment_status_id, code, name, is_final, display_order	SMALLINT, NVARCHAR, BIT	No	Identificadores 1 a 5; 3 = APPROVED
payment.payment	payment_id, booking_id, payment_account_id, payment_status_id	BIGINT, SMALLINT	No	FK a payment_account y payment_status
	amount	DECIMAL(12,2)	No	> 0; monto a pagar de la reserva
	processed_at, approved_by, rejection_reason	DATETIME2(3), BIGINT, NVARCHAR(200)	Sí	Un pago aprobado por reserva (ux_payment_one_approved_per_booking)
payment.payment_receipt	payment_receipt_id, payment_id	BIGINT	No	FK a payment
	file_url	NVARCHAR(MAX)	No	Imagen del comprobante como URL de datos (changeset 016)
	transaction_reference, reported_amount	NVARCHAR(100), DECIMAL(12,2)	Sí	Referencia indexada; su reutilización la rechaza el servicio
	uploaded_at, uploaded_by	DATETIME2(3), BIGINT	No	—
	reviewed_at, reviewed_by, review_comment	DATETIME2(3), BIGINT, NVARCHAR(300)	Sí	Ambos o ninguno (ck_preceipt_review)
payment.loyalty_movement_type	loyalty_movement_type_id, code, name, sign, display_order, is_active	SMALLINT, NVARCHAR	No	EARNED, REDEEMED, EXPIRED, ADJUSTED, REVERSED (changeset 020)
payment.loyalty_transaction	loyalty_transaction_id, customer_id, loyalty_movement_type_id, booking_id, points, balance_after, expires_on, description	BIGINT, SMALLINT, INT, DATE, NVARCHAR(200)	No/Sí	FK a payment.loyalty_movement_type; customer_id guarda el user_id; EARNED una vez por cliente y reserva
	sequence_no	INT	No	Posición del movimiento en el libro del cliente; única por cliente (changeset 020)

Nota. Fuente: changelogs 001 a 020 de payment-service. Cada tabla tiene también el bloque de auditoría.

Tabla 17

Diccionario de Datos del Esquema de Notificaciones

Tabla	Campo	Tipo	Nulo	Restricción o descripción
notification.notification_type	notification_type_id, code, name, display_order, is_active	SMALLINT, NVARCHAR	No	26 tipos (de cliente, de personal, de pago, de fidelización, de inspección, de reserva sin operario, de versión nueva de la app, de servicio sin registrar, changelog 041, y de sistema)
notification.notification	notification_id, user_id, notification_type_id, title, message, reference_entity, reference_id, is_read, read_at, sent_at	BIGINT, SMALLINT, NVARCHAR(120), NVARCHAR(500), BIT, DATETIME2(3)	No/Sí	FK a notification_type; borrado lógico
notification.device_token	Token push de Expo de cada dispositivo de un usuario	—	—	Changeset 016
notification.processed_event	eventId de cada evento consumido (idempotencia)	—	—	Changeset 017
notification.booking_reminder	Un recordatorio pendiente por reserva y anticipación (usuario, hora de la reserva, remind_at, sent_at)	—	—	Changeset 019
notification.booking_tracking	booking_id, booking_code, customer_user_id, operator_user_id, scheduled_start, is_cancelled, updated_at	BIGINT, NVARCHAR(20), DATETIME2(3), BIT	No/Sí	Proyección propia para distinguir una reserva nueva de una reprogramada y conocer al operario asignado (changeset 027)

Nota. Fuente: modelo de datos y changelogs de notification-service. Las columnas de device_token, processed_event y booking_reminder se deben completar desde sus changelogs (Anexo B).

11.4 Reglas de Integridad

Tabla 18

Reglas de Integridad y Dónde se Garantizan

Regla	Dónde se garantiza
Dos reservas activas no pueden ocupar la misma bahía en horarios superpuestos.	Disparador tr_booking_bay_overlap y booking-service.
Un vehículo no puede tener dos reservas superpuestas; una reserva no puede quedar fuera del horario, del descanso ni de las excepciones.	booking-service.
El precio de una línea de reserva no cambia si cambia el catálogo.	Disparador tr_service_price_immutable.
Un motivo de cancelación solo existe con el estado CANCELLED.	Restricción ck_booking_cancel.
En las reservas de un cliente solo se usan sus vehículos.	booking-service mediante customer-service (404 VEHICLE_NOT_FOUND).
El operario asignado debe estar activo, de turno durante toda la reserva, no ausente y no ocupado.	AssignmentPolicy en operations-service.
Una asignación manual y una automática simultáneas no le dan al mismo operario dos servicios a la misma hora.	Candado *operations-assignment* (*sp_getapplock*) en operations-service.
Solo el operario asignado inicia o finaliza; se puede iniciar desde 15 minutos antes de la hora de la reserva y hasta 4 horas después de su hora de fin (*START_LATE_LIMIT*, desde el 24 de octubre de 2026), para registrar tarde un servicio que sí se hizo (*StartPolicy*, *SERVICE_TOO_EARLY*, *SERVICE_TOO_LATE*; la respuesta trae *startableAt* y *startableUntil*); un servicio a la vez. Iniciar, terminar y cerrar guardan la ejecución antes de llamar a booking-service y, si booking responde 409 porque el cambio ya se aplicó, lo dan por hecho (OPS-01); las ejecuciones se bloquean por fila (OPS-03).	operations-service (y booking-service desde *develop* del 23 de octubre de 2026, E-01).
Una reserva se califica una vez, solo cuando está COMPLETED y solo por su dueño; la calificación se escribe en cada línea.	operations-service y restricciones ck_sexec_*.
Como máximo un pago aprobado por reserva; ningún segundo pago mientras uno esté en revisión; el monto es el monto a pagar de la reserva.	Índice ux_payment_one_approved_per_booking y payment-service.
Una referencia de transacción no se puede reutilizar.	payment-service (409 TRANSACTION_REFERENCE_REUSED).
El comprobante del cliente pesa como máximo 2 MB.	Dominio de payment-service (*Payment.MaxReceiptBytes*, 400 RECEIPT_TOO_LARGE).
Un cupón se canjea solo si está activo, en sus fechas y desbloqueado por el saldo; una vez por reserva.	payment-service (CustomerPromotionService) y promotion.booking_promotion.
Dos movimientos del libro de puntos del mismo cliente no se pueden calcular sobre el mismo saldo.	sequence_no única por cliente (409 en caso de conflicto).

Nota. Elaborado por los autores a partir del código y de los modelos de datos de los servicios.

11.5 Migraciones y Datos Iniciales

Cada servicio es dueño del changelog de Liquibase de sus esquemas y lo aplica al arrancar; luego *ddl-auto: validate* comprueba que las entidades JPA coincidan con las tablas. Cada servicio registra su historial en sus propias tablas `DATABASECHANGELOG_<SERVICIO>` y `DATABASECHANGELOGLOCK_<SERVICIO>`. payment-service es la excepción: sus changelogs están en *db/changelog* y los ejecuta el contenedor *payment-migrate* (ADR-013). Todo changeset tiene una precondición *onFail:MARK_RAN*, de modo que una base de datos creada con el script global *lavarapido-6-services-sqlserver.sql* converge con una creada por los servicios. La numeración es compartida (001 a 013 esquemas iniciales, 014 en adelante cambios posteriores, 101 en adelante semillas), salvo en payment-service, que usa su propia numeración (TD-019; va en la 023). Al 26 de octubre de 2026, el número más alto de la numeración global es 049 (*049-booking-payment-mode-deadline.sql* de booking-service: *payment_mode*, *payment_deadline*, *payment_reported_at* y el índice filtrado *ix_booking_payment_pending*); el siguiente changeset compartido es el 050. La semilla más

reciente es la 117 de booking-service (motivo de cancelación *PAYMENT_TIMEOUT*); la siguiente es la 118. La lista de changelogs de cada servicio está en el Manual de Instalación, sección 11.2.

12. Módulos Técnicos

12.1 security-service

Función: identidad y acceso: registro con la aceptación de los documentos legales, ingreso con JWT y bloqueo por intentos fallidos, sesiones, recuperación de la contraseña por código, perfil (nombre, teléfono, dirección, correo, contraseña y desactivación), preferencias (tema, idioma y los tres interruptores de notificación, con actualización parcial), administración de cuentas y la matriz informativa de permisos de los tres roles fijos.

Archivos principales: AuthController (incluye *GET /api/v1/auth/session*), ActiveSessionValidator, UserProfileController, UserAdministrationController, InternalUserController y el controlador de roles y permisos; AuthenticationService, AccountRegistrationService, PasswordRecoveryService, EmailChangeService, AccountDeactivationService, UserAdministrationService, UserPreferenceService; PasswordPolicy; JwtTokenIssuer; LoginLockout y LoginLockoutPersistenceAdapter; UserAddressController; EmailResetCodeNotifier; RabbitDomainEventPublisher; DevDataSeeder.

Tablas: esquemas security y audit.

Reglas: contraseñas con BCrypt (costo 12); política de contraseñas de 8 a 72 caracteres con una mayúscula, un dígito y un carácter especial, sin espacios; el código de recuperación tiene 6 dígitos, dura 15 minutos, es de un solo uso y cinco intentos fallidos lo invalidan (contador en memoria); solicitar un código nuevo invalida el anterior; el correo de una cuenta inexistente produce la misma respuesta; el cambio de correo y la desactivación requieren la contraseña actual; un administrador no puede desactivar su propia cuenta (403 CANNOT_DISABLE_OWN_ACCOUNT) y una cuenta de administrador no se desactiva desde la administración de usuarios (403 ADMIN_CANNOT_BE_DISABLED); el rol se asigna al crear la cuenta y no hay un endpoint para cambiarlo; los identificadores de permisos desconocidos se rechazan con 400 UNKNOWN_PERMISSION; cinco intentos fallidos en 15 minutos bloquean la cuenta durante 15 minutos (*LoginLockout*, 423 ACCOUNT_LOCKED) y un ingreso correcto o el restablecimiento de la contraseña limpian el contador; el registro guarda la fecha de aceptación de los documentos legales cuando llegan *acceptsTerms* y *acceptsDataPolicy*; cada token lleva en el claim *sid* su sesión, y desde el 21 de octubre de 2026 *ActiveSessionValidator* rechaza el token de una sesión cerrada (logout, cuenta inhabilitada por el administrador o desactivada por el dueño) en todas las rutas de security-service; *GET /api/v1/auth/session* responde 204 si la sesión sigue abierta y 401 si no, para el gateway.

Eventos: publica UserRegistered (*security.user_registered*), que lleva *createdByAdmin* para que el mensaje de bienvenida explique cómo ingresar; PasswordChanged, EmailChanged y AccountDeactivated los levanta el dominio. **Rutas internas:** contacto de un usuario con sus interruptores y lista de los administradores activos.

12.2 customer-service

Función: perfil del cliente y vehículos.

Archivos principales: CustomerVehicleController, VehicleTypeController, AdminVehicleController; CustomerVehicleService, CustomerRegistrationService (consume UserRegistered), VehicleTypeQueryService; LicensePlatePolicy y LicensePlate.

Tablas: customer.vehicle_type, customer y customer_vehicle.

Reglas: el perfil se crea a partir del evento UserRegistered o en la primera solicitud del cliente (leyendo el *personId* de security-service); las placas se guardan limpias y en mayúsculas, con el formato ABC123 para automóviles y ABC12D para

motocicletas; las placas son únicas entre los vehículos activos; el vehículo de otro cliente responde 404. **Eventos:** consume *security.user_registered* (cola *customer-service.events*); sus eventos de dominio solo se registran en el log.

12.3 booking-service

Función: catálogo, precios y puntos por tipo de vehículo, establecimiento, horario semanal con descanso, excepciones, bahías, disponibilidad y reservas.

Reglas: la disponibilidad se calcula sobre una rejilla de 30 minutos dentro del horario de atención, fuera del descanso, no en el pasado, hasta 60 días adelante y con al menos una bahía activa libre durante toda la duración; una reserva se crea directamente como CONFIRMED con una bahía; si el turno no se puede reservar, no se guarda nada y la respuesta 409 SLOT_UNAVAILABLE trae hasta cinco horas alternativas del mismo día y el motivo (*reason*: CLOSED_DAY, OUTSIDE_HOURS, TIME_PASSED o NO_FREE_BAY, calculado por *AvailabilityPolicy.whyUnavailable*); la solicitud puede traer *preferredOperatorId*, que viaja en BookingConfirmed para que operations-service lo respete si está libre; los clientes pueden reprogramar o cancelar solo antes del inicio; el administrador pasa las reservas por IN_PROGRESS, COMPLETED, NO_SHOW o CANCELLED; cada servicio del catálogo tiene sus *loyalty_points*, cada línea de reserva los copia cuando se construye la reserva y *Booking.totalLoyaltyPoints()* expone el total en *BookingResponse.totalLoyaltyPoints*; los cambios del horario, las excepciones y las bahías se registran en *schedule_history*; los datos del negocio del establecimiento (identidad, información tributaria y canales) se leen y guardan con *GET* y *PUT /api/v1/admin/establishment*; las rutas */internal/v1/bookings* sirven a operations-service con la llave interna; crear, reprogramar, cerrar reservas vencidas y marcar una reserva sin operario como NO_SHOW toman el candado de la agenda (*sp_getapplock* sobre *booking-schedule*). **Reservas vencidas (desde el 21 de octubre de 2026):** cada 5 minutos *ExpiredBookingsJob* llama a *BookingService.closeExpired*, que cancela solo las reservas que siguen SCHEDULED una hora después de su hora de fin (*BOOKING_EXPIRY_GRACE_MINUTES*), con el motivo del sistema *EXPIRED* (semilla 112), que no aparece en la lista de motivos. Una CONFIRMED nunca se cierra desde booking-service, porque pudo atenderse sin registrarse: la decide operations-service (sección 12.4). Para la que nunca tuvo operario, operations-service llama a la ruta interna *POST /internal/v1/bookings/{id}/no-show* (*BookingService.markUnattended*), que vuelve a comprobar que la reserva está CONFIRMED y ya venció (409 BOOKING_NOT_EXPIRED si no) y la pasa a NO_SHOW. Nada se reembolsa solo. La tarea se apaga con *BOOKING_EXPIRY_CHECK_ENABLED=false* y su intervalo es *BOOKING_EXPIRY_CHECK_INTERVAL*. **Eventos:** publica BookingConfirmed (creada o reprogramada) y BookingCancelled (también al cancelar una reserva vencida; el paso a NO_SHOW no publica evento).

Pago al reservar (desde el 26 de octubre de 2026): el cliente debe enviar *paymentMode* (*DIGITAL* o *CASH*) al crear la reserva (400 *INVALID_PAYMENT_MODE*); el administrador no está obligado, y sus reservas no tienen plazo ni límite. *PaymentPolicy* aplica dos reglas: como máximo *BOOKING_CASH_ACTIVE_LIMIT* (1) reservas activas (SCHEDULED, CONFIRMED o IN_PROGRESS) en efectivo hechas por el propio cliente (*booked_by*), contadas con la agenda bloqueada (*lockSchedule*; 409 *CASH_BOOKING_LIMIT*), y, para *DIGITAL*, *paymentDeadline* = min(creación + *BOOKING_PAYMENT_WINDOW* (30 min), inicio). *PaymentEventsListener* consume *payment.reported* y *payment.confirmed* y marca *payment_reported_at* (*Booking.markPaymentReported*, idempotente). *PaymentDeadlineJob* corre cada *BOOKING_PAYMENT_EXPIRY_INTERVAL* (60 s; se apaga con *BOOKING_PAYMENT_EXPIRY_ENABLED=false*) y, antes de cancelar una reserva vencida (*Booking.paymentOverdue*), le pregunta a payment-service: con *reported=true* la marca y no la cancela; con *false* la cancela con *PAYMENT_TIMEOUT* y publica *BookingCancelled*; si payment no responde, la deja para la siguiente vuelta (*PaymentTimeoutOutcome*: CANCELLED,

PAYMENT_REPORTED o NOT_OVERDUE). Reprogramar no cambia el modo ni el plazo, y las reservas anteriores tienen *payment_mode* NULL y ninguna de estas reglas.

12.4 operations-service

Función: operarios, turnos semanales, ausencias, perfil y habilidades del operario, asignación manual y automática, ejecución, calificaciones e informes de inspección (ADR-014).

Reglas: las filas de operarios se crean a partir de las cuentas con el rol OPERATOR; si dos solicitudes crean el mismo operario a la vez, la segunda vuelve a leer la fila ya creada en lugar de fallar; *AssignmentPolicy* devuelve el motivo que bloquea a un operario (OPERATOR_INACTIVE, OPERATOR_NOT_ON_SHIFT, OPERATOR_ABSENT, OPERATOR_BUSY); solo se asignan reservas SCHEDULED o CONFIRMED, y se crea una *service_execution* por línea de reserva. **Asignación automática (RF-008):** al consumir BookingConfirmed, *AutoAssignmentPolicy* elige al operario que pidió el cliente si sigue libre y, si no, al libre con menos reservas ese día (a igualdad, el de id menor); si nadie está libre, la reserva queda sin operario para el administrador, y una reprogramación conserva al operario que ya tenía. *GET /api/v1/operators/available* lista los operarios libres que el cliente puede elegir al reservar. Un servicio se puede iniciar desde 15 minutos antes de la hora reservada (*StartPolicy*); iniciar y finalizar mueven la reserva en booking-service; las asignaciones que devuelve *GET /api/v1/admin/assignments* incluyen la bahía de la reserva (ADR-015). Desde el 21 de octubre de 2026 la asignación manual y la automática toman el mismo candado (*ExecutionRepository.lockAssignments*, *sp_getapplock* sobre *operations-assignment*) antes de leer las reservas del operario y guardar, y *GET /api/v1/admin/assignments/unassigned* devuelve *{count, bookingIds}*, el único conteo de reservas sin operario (SCHEDULED o CONFIRMED, sin operario en ninguna línea y cuya hora no ha terminado) que usan el panel y la pantalla de reservas de la web y de la app móvil. **Reservas que terminaron sin cerrarse (desde el 21 de octubre de 2026):** cada 5 minutos *FollowUpJob* llama a *FollowUpService.followUp*, que revisa las reservas de los últimos 7 días con *FollowUpPolicy*. La política solo decide sola cuando está segura: NO_OPERATOR (CONFIRMED sin operario, una hora después de su fin) se cierra como NO_SHOW mediante la ruta interna de booking-service; NOT_STARTED (CONFIRMED con operario, sin iniciar cuando terminó su hora) y NOT_FINISHED (IN_PROGRESS sin finalizar una hora después de su fin) nunca se marcan solas y quedan "por cerrar". Para estas dos, operations-service publica una sola vez ServiceNotStarted o ServiceNotFinished y guarda la hora del aviso en *execution.service_execution.follow_up_notice_at* (changelog 040), que se borra al iniciar el servicio. *GET /api/v1/admin/assignments/to-close* lista las reservas por cerrar (por defecto, los últimos 28 días, *FOLLOW_UP_LOOK_BACK_DAYS*) y *POST /api/v1/admin/assignments/{bookingId}/attended* (*FollowUpService.closeAsAttended*) cierra una como atendida: las líneas abiertas quedan COMPLETED sin inventar horas (sin hora de inicio no cuenta para la puntualidad y sin hora de fin no suma horas trabajadas, pero sí cuenta como servicio hecho y como ingreso), la reserva avanza a IN_PROGRESS si hacía falta y luego a COMPLETED, y no se publica ServiceCompleted, para que al cliente no le llegue "ya puedes recogerlo" horas después; el cliente sí puede calificar. Si la reserva no está por cerrar, responde 409 BOOKING_NOT_TO_CLOSE. El "No asistió" del administrador sigue siendo el cambio de estado de booking-service (*PATCH /api/v1/admin/bookings/{id}/status*) y solo aplica a una reserva que nunca se inició. Las listas del operario ya no muestran reservas NO_SHOW ni CANCELLED. La tarea usa *FOLLOW_UP_ENABLED*, *FOLLOW_UP_INTERVAL* y *BOOKING_EXPIRY_GRACE_MINUTES*, y necesita *INTERNAL_API_KEY*. **Inspección (RF-036):** solo para servicios de 180 minutos o más; fases Recepción, Exterior, Interior, Motor y Entrega; hasta cinco fotos de 2 MB por hallazgo, guardadas en *execution.inspection_photo*; el enlace público usa un token aleatorio y solo funciona después de publicar. **Eventos:** consume *booking.confirmed* y publica OperatorAssigned, ServiceStarted, ServiceCompleted, InspectionShared, ServiceNotStarted y ServiceNotFinished.

12.5 payment-service

Función: cuentas de pago con QR, pagos con comprobante, revisión, reembolsos, pagos manuales, promociones y puntos de fidelización (ADR-008, ADR-013 y ADR-015).

Reglas: solo se pueden pagar reservas CONFIRMED, IN_PROGRESS o COMPLETED, y *PaymentPolicy.CanReportPayment* exige además que no haya un pago aprobado ni uno pendiente o en revisión (expuesto como *payable* en *GET /api/v1/payments/me/bookings*); el monto a pagar es el total que devuelve booking-service menos el cupón canjeado (*AmountDueCalculator*); el monto del pago lo calcula el servidor (*AmountDueCalculator*, en *PaymentCommandService* al reportar y al registrar un pago en persona), no la pantalla; lo que el cliente dice que pagó se guarda aparte en el comprobante (*reported_amount*) para que el administrador lo compare, y el pago pasa directamente a IN_REVIEW; una referencia de transacción reutilizada se rechaza; el comprobante pesa como máximo 2 MB (400 RECEIPT_TOO_LARGE; la web y la app lo comprimen antes de enviarlo); *GET /api/v1/payment-accounts* devuelve las cuentas activas, incluida la de efectivo mientras está activa; el rechazo exige un motivo de hasta 200 caracteres; un pago manual queda aprobado de inmediato; la máquina de estados es PENDING → IN_REVIEW → APPROVED o REJECTED, y APPROVED → REFUNDED. **Fidelización y promociones (ADR-015):** una aprobación acredita los puntos de la reserva como EARNED (idempotente por cliente y reserva); un reembolso los revierte (REVERSED); el saldo es el *balance_after* del último movimiento; un cupón es un descuento porcentual que se desbloquea cuando el saldo alcanza *required_points*, y canjearlo no consume puntos. **Eventos:** publica PaymentConfirmed, PaymentRejected, PaymentRefunded, PromotionRedeemed y LoyaltyPointsEarned después del commit, solo con *MESSAGING_ENABLED=true*. **Correlación y salud:** *CorrelationIdMiddleware* toma o crea el *X-Correlation-Id* y lo reenvía a booking-service; */health* y */actuator/health* comprueban SQL Server con un límite de 3 segundos.

Pago al reservar (desde el 26 de octubre de 2026): cuando el cliente reporta un pago con comprobante (*POST /api/v1/payments*, *Payment.ReportWithReceipt*), también al volver a reportarlo después de un rechazo, payment-service publica *PaymentReported* (*payment.reported*; payload *paymentId*, *bookingId*, *customerUserId*, *amount*). La ruta interna *GET /internal/v1/payments/bookings/{bookingId}/reported* responde *{"bookingId":123,"reported":true}*, con *reported* verdadero si hay un pago IN_REVIEW o APPROVED; la protege *InternalApiKeyAuthenticationHandler* (*X-Internal-Key = INTERNAL_API_KEY*; 401 sin llave o con una errada, y un JWT de usuario no sirve). Hasta ese día payment-service no validaba ninguna llave interna. Reportar o aprobar el pago de una reserva cancelada sigue respondiendo 409 *BOOKING_NOT_PAYABLE* (*BookingPaymentGuard.EnsureBookingIsPayable*).

12.6 notification-service

Función: convierte eventos en notificaciones (bandeja, push mediante Expo y correo) y programa los recordatorios de las reservas (ADR-011).

Reglas: la notificación siempre se guarda primero, y una falla del push nunca hace fallar la solicitud; el consumo es idempotente por *eventId* (*processed_event*); BookingConfirmed programa recordatorios 24 h y 1 h antes, BookingCancelled los cancela y una reprogramación los mueve; los correos al cliente leen la dirección vigente y los interruptores desde security-service mediante la ruta interna; un aviso se envía por correo solo cuando cambia algo para el cliente (reserva, recordatorio, resultado de pago, cupón e informe de inspección), mientras que el avance del servicio, los puntos ganados y todos los avisos al personal se quedan en la bandeja y en push; la proyección *booking_tracking* alimenta los avisos al personal (reprogramada, cancelada y desasignada para el operario; creada, reprogramada y cancelada para todo administrador activo); el endpoint del administrador acepta solo

SYSTEM_MESSAGE e *INSPECTION_REPORT*; InspectionShared genera el aviso al cliente con el enlace público del informe (*/report/{token}*); cada pocos minutos *UnassignedBookingAlertService* avisa a los administradores de las reservas confirmadas que empiezan en menos de 2 horas y siguen sin operario (una vez por reserva); la ruta interna *POST /internal/v1/notifications/app-update*, que llama *deploy/publish-apk.sh*, avisa por push que hay una versión nueva de la aplicación; desde el 21 de octubre de 2026, ServiceNotStarted y ServiceNotFinished (*execution.service_not_started* y *execution.service_not_finished*) generan *OPERATOR_SERVICE_NOT_REGISTERED* para el operario ("¿Atendiste este servicio?") y *ADMIN_SERVICE_NOT_REGISTERED* para cada administrador activo ("Servicio sin registrar"), solo en la bandeja y en push, con los tipos sembrados en el changelog 041, y un campo *null* en el payload de un evento se ignora en lugar de mandar el mensaje entero a la DLQ. **Configuración:** MESSAGING_ENABLED, PUSH_ENABLED, MAIL_ENABLED, INTERNAL_API_KEY, REMINDERS_ENABLED, REMINDER_LEADS y REMINDER_CHECK_INTERVAL.

12.7 Clientes Web y Móvil

La aplicación web consume el gateway mediante los servicios de *core/services*; *auth-interceptor* agrega el token y redirige a */auth?expired* cuando el servidor responde 401 mientras la sesión sigue abierta (una respuesta 401 que llega después de cerrar sesión se ignora, para no mostrar "Tu sesión expiró" a quien salió por su cuenta), y *api-error* traduce los códigos de error. Desde el 21 de octubre de 2026 el servicio *auth* escucha el evento *storage*: si otra pestaña del mismo navegador cierra sesión, esta vuelve al inicio de sesión, y si entra otra cuenta, se recarga para no mostrar datos de la anterior; la pantalla de pagos comprime el comprobante a máximo 2 MB y amplía el QR con una lupa. Todas las pantallas leen y guardan mediante la API: los datos del negocio usan *BusinessStore* sobre booking-service y los informes de inspección usan *inspection-api* sobre operations-service (las fotos se comprimen en el navegador antes de enviarse). Solo los reportes se calculan en el navegador a partir de las reservas y los pagos; desde el 21 de octubre de 2026 cuentan como servicios realizados solo las reservas completadas. Los roles y permisos, las promociones, la fidelización, las preferencias y los interruptores de notificación se leen y se guardan mediante la API (*role-permissions-api*, *promotions-api*, *loyalty-api* y *preferences*). La aplicación móvil cubre los tres roles con pantallas equivalentes y guarda la sesión con *expo-secure-store*. Su cliente HTTP (*core/api/httpClient.ts*) envía JSON o formularios *multipart* (las fotos de la inspección), admite un tiempo de espera propio por petición (60 segundos para subir fotos) y aplica la misma regla de la web ante un 401. *core/api/config.ts* expone *WEB_URL*, la dirección de la web que se usa para armar el enlace público del informe; se toma de *EXPO_PUBLIC_WEB_URL* o, si no está, del mismo equipo del gateway con el puerto 4200. Desde la noche del 21 de octubre de 2026 el inicio del administrador, en la web (*reservations-store*) y en la app, muestra la tarjeta "Por cerrar" con los botones "Se atendió" y "No asistió" solo cuando hay reservas por cerrar, y las dos usan *GET /api/v1/admin/assignments/unassigned* para el conteo de reservas sin operario (si operations-service no responde, aplican la misma regla con los datos que tienen). En la app, además, el formulario de reserva del administrador guarda el operario y el estado elegidos, la barra de puntos del cliente queda completa cuando no queda nada por desbloquear y no muestra "0% OFF", el operario ve "—" en la calificación cuando no tiene calificaciones y su inicio no repite el contador "En progreso", la pestaña de roles dice que los tres roles son fijos y los permisos son de referencia, y los avisos nuevos abren la pantalla de servicios del operario o el inicio del administrador.

12.8 Ciclos de Vida

Tabla 19

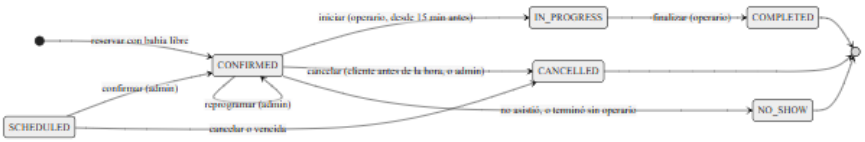
Máquinas de Estado del Dominio

Entidad	Estados (códigos de la base de datos)	Transiciones principales
Reserva	SCHEDULED, CONFIRMED, IN_PROGRESS, COMPLETED, CANCELLED, NO_SHOW	Creación → CONFIRMED; SCHEDULED → CONFIRMED; CONFIRMED → IN_PROGRESS → COMPLETED; SCHEDULED o CONFIRMED → CANCELLED o NO_SHOW. El sistema solo cierra una reserva cuando está seguro, una hora después de su fin: SCHEDULED → CANCELLED con el motivo EXPIRED y CONFIRMED sin operario → NO_SHOW. Una CONFIRMED con operario que nadie registró no cambia sola: la resuelve el administrador (Se atendió → IN_PROGRESS → COMPLETED, o No asistió).
Ejecución del servicio	PENDING, IN_PROGRESS, PAUSED, COMPLETED, WITH_ISSUE	PENDING → IN_PROGRESS → COMPLETED. PAUSED y WITH_ISSUE existen en el catálogo, pero no se usan.
Pago	PENDING, IN_REVIEW, APPROVED, REJECTED, REFUNDED	PENDING → IN_REVIEW → APPROVED o REJECTED; APPROVED → REFUNDED.
Movimiento de puntos	EARNED, REVERSED (también REDEEMED, EXPIRED y ADJUSTED en el catálogo)	Aprobación del pago → EARNED; reembolso → REVERSED. Los movimientos no se modifican.

Nota. Elaborado por los autores a partir del código y de los modelos de datos de los servicios.

Figura 3

Diagrama de Estados de la Reserva



Nota. Los estados en gris son finales. El administrador también puede cambiar el estado de una reserva directamente desde el panel (*PATCH /api/v1/admin/bookings/{id}/status*). *Vencida* es el cierre automático de *ExpiredBookingsJob* (sección 12.3) y *terminó sin operario* lo pide *FollowUpJob* de operations-service (sección 12.4). Una reserva confirmada con operario que nadie registró no cambia de estado sola: la resuelve el administrador desde "Por cerrar" (Se atendió → IN_PROGRESS → COMPLETED, o No asistió).

13. API o Rutas del Sistema

La API externa sigue el estilo REST (Fielding, 2000). Todo acceso externo pasa por el gateway. Los contratos OpenAPI de los siete componentes están en *07-api/contracts/openapi* de la documentación, y cada servicio Java publica su Swagger UI en el perfil *dev* (*/swagger-ui.html*); payment-service lo publica en */swagger* en el entorno Development.

13.1 Convenciones Generales

- **Versionado:** prefijo */api/v1* en la ruta.
- **Formato:** JSON; identificadores numéricos (IDENTITY), no UUID.

- **Autenticación:** encabezado *Authorization: Bearer `<JWT>`* en toda ruta no pública.
- **Correlación:** encabezado *X-Correlation-Id*.
- **Propiedad:** la identidad proviene del token verificado, nunca del cuerpo; los precios, los totales, los descuentos y las horas de finalización nunca se aceptan del cliente.
- **Paginación:** las listas paginadas (cuentas y notificaciones) devuelven un objeto *PageResponse* con los elementos y los datos de la paginación.
- **Errores:** problem details del RFC 9457 con un *code* estable (sección 9.5.3).
- **Rutas internas:** */internal/v1/** con el encabezado *X-Internal-Key*; el gateway no las enruta.

13.2 Autenticación en la API

security-service emite un JWT firmado con HS256 (secreto JWT_SECRET de al menos 32 bytes) válido durante una hora, con los claims *sub* (identificador del usuario), *roles*, *sid* (identificador de la sesión), *iss* (lavarapido-security-service), *aud* (lavarapido-api), *iat*, *nbf*, *exp* y *jti*. No hay token de actualización: cuando el token vence, la persona ingresa de nuevo. Un JWT no se puede revocar por sí mismo antes de su vencimiento (Jones et al., 2015). Desde el 21 de octubre de 2026 el sistema lo compensa con el claim *sid*: cerrar la sesión, o que el administrador inhabilite la cuenta o el dueño la desactive, cierra el registro de la sesión; security-service rechaza los tokens de esa sesión y el gateway se lo pregunta (*GET /api/v1/auth/session*) antes de pasar cualquier petición a los demás servicios, con la respuesta guardada 30 segundos por token (sección 9.2). Todo servicio valida la firma, el emisor y la audiencia con el mismo secreto.

13.3 Endpoints

Tabla 20

Endpoints del Sistema

Método	Ruta	Rol	Descripción
POST	/api/v1/auth/register	Público	Registra un cliente (documentNumber, firstName, lastName, email, phone, password, acceptsTerms, acceptsDataPolicy).
POST	/api/v1/auth/login	Público	Devuelve accessToken, vencimiento y usuario; 423 ACCOUNT_LOCKED después de cinco intentos fallidos.
POST	/api/v1/auth/logout	Token	Cierra la sesión; el token deja de servir.
GET	/api/v1/auth/session	Token	204 si la sesión del token sigue abierta; 401 si se cerró. La usa el gateway.
POST	/api/v1/auth/password/forgot, /verify, /reset	Público	Recuperación de la contraseña en tres pasos.
GET, PATCH	/api/v1/users/me	Token	Perfil.
PUT	/api/v1/users/me/email, /api/v1/users/me/password	Token	Cambiar el correo o la contraseña (requiere la contraseña actual).
POST	/api/v1/users/me/deactivate	Token	Desactiva la propia cuenta.
GET, PUT	/api/v1/users/me/address	Token	Dirección de contacto del perfil.
GET, PUT	/api/v1/users/me/preferences	Token	Tema, idioma y los interruptores notificationsEnabled (push), emailRemindersEnabled y promotionsEnabled; PUT es parcial.
GET, POST	/api/v1/admin/users	ADMIN	Lista (paginada, por rol) y crea cuentas.
PATCH	/api/v1/admin/users/{userId}/status	ADMIN	Activa o desactiva una cuenta.
GET	/api/v1/admin/roles-permissions	ADMIN	Catálogo de permisos y los permisos de cada rol fijo.

Método	Ruta	Rol	Descripción
PUT	/api/v1/admin/roles-permissions/{role}	ADMIN	Reemplaza los permisos de un rol (400 UNKNOWN_PERMISSION).
GET	/api/v1/vehicle-types	Público	Tipos de vehículo.
GET, POST	/api/v1/vehicles	CLIENT	Vehículos propios; registra un vehículo.
GET, PUT, DELETE	/api/v1/vehicles/{id}	CLIENT	Vehículo propio (404 si pertenece a otro cliente).
GET	/api/v1/admin/vehicles?ids= o ?plate=	ADMIN	Vehículos con su ownerUserId.
GET	/api/v1/catalog/categories, /api/v1/catalog/services?vehicleTypeId=	Público	Catálogo con precios por tipo de vehículo y puntos.
GET	/api/v1/schedule/business-hours, /api/v1/schedule/exceptions, /api/v1/establishment	Público	Horario y datos del lavadero.
GET	/api/v1/bookings/availability?date=&vehicleTypeId=&serviceIds=	Token	Horas de inicio libres de un día.
POST	/api/v1/bookings	CLIENT	Crea una reserva.
GET	/api/v1/bookings/me, /api/v1/bookings/{id}, /api/v1/bookings/cancellation-reasons	CLIENT	Reservas propias y catálogo de motivos (sin el motivo del sistema EXPIRED).
PUT, POST	/api/v1/bookings/{id}, /api/v1/bookings/{id}/cancel	CLIENT	Reprogramar o cancelar antes del inicio. La web y la app solo usan la cancelación (sin motivo: se guarda CUSTOMER_REQUEST); la reprogramación la hace el administrador.
GET, POST	/api/v1/admin/bookings (?from=&to=&status=)	ADMIN	Lista y crea reservas para cualquier cliente.
GET	/api/v1/admin/bookings/day-summary?date=	ADMIN	Indicadores del día para el panel.
GET, PUT	/api/v1/admin/bookings/{id}	ADMIN	Detalle y reprogramación.
PATCH	/api/v1/admin/bookings/{id}/status	ADMIN	Cambia el estado (con reasonCode al cancelar).
GET, POST, PUT, DELETE, PATCH	/api/v1/admin/catalog/services[/{id}[/status]]	ADMIN	Servicios, precios y puntos.
PUT	/api/v1/admin/schedule/business-hours	ADMIN	Horario semanal.
GET	/api/v1/admin/schedule/history	ADMIN	Historial de cambios del horario, las excepciones y las bahías.
GET, PUT	/api/v1/admin/establishment	ADMIN	Datos del negocio del establecimiento.
POST, PUT, DELETE	/api/v1/admin/schedule/exceptions[/{id}]	ADMIN	Excepciones.
GET, POST, PUT, DELETE	/api/v1/admin/bays[/{id}]	ADMIN	Bahías.
GET, PATCH	/api/v1/operator/bookings[/{id}[/status]]	OPERATOR, ADMIN	Las usa operations-service.
GET	/api/v1/admin/operators	ADMIN	Operarios.
GET, PUT	/api/v1/admin/operators/profiles, /api/v1/admin/operators/{id}/profile	ADMIN	Especialidad y habilidades de los operarios.
GET	/api/v1/operators/available?date=&time=&minutes=	Token	Operarios libres que el cliente puede elegir al reservar.
PATCH, PUT	/api/v1/admin/operators/{id}/status, /availability	ADMIN	Activar o desactivar; reemplazar el turno semanal.
GET, POST, DELETE	/api/v1/admin/operators/{id}/absences[/{absenceId}]	ADMIN	Ausencias.
GET	/api/v1/admin/assignments (?from=&to=)	ADMIN	Asignaciones con la bahía de cada reserva.

Método	Ruta	Rol	Descripción
GET	/api/v1/admin/assignments/unassigned (?from=&to=)	ADMIN	*{count, bookingIds}* de las reservas programadas o confirmadas sin operario que todavía no terminan; el panel y la pantalla de reservas de la web y de la app usan este conteo.
GET	/api/v1/admin/assignments/to-close (?from=&to=)	ADMIN	Reservas con operario que terminaron sin registrarse (NOT_STARTED o NOT_FINISHED); por defecto, los últimos 28 días (*FOLLOW_UP_LOOK_BACK_DAYS*).
POST	/api/v1/admin/assignments/{bookingId}/attended	ADMIN	Cierra como atendida una reserva por cerrar, sin inventar horas (409 BOOKING_NOT_TO_CLOSE si no lo está).
PUT	/api/v1/admin/assignments/{bookingId}	ADMIN	Asigna o cambia el operario (409 con el motivo).
GET	/api/v1/admin/assignments/{bookingId}/candidates	ADMIN	Operarios con su motivo de bloqueo.
GET, POST, PATCH, DELETE	/api/v1/admin/inspections/{bookingId}[/phases/{phase}, /findings[/{findingId}], /publish, /share, /photos/{photoId}]	ADMIN	Informe de inspección: iniciar, fases, hallazgos con fotos (multipart), publicar y avisar al cliente.
GET	/api/v1/public/inspections/{token}[/photos/{photoId}]	Público	Informe publicado y sus fotos, sin iniciar sesión.
GET	/api/v1/operator/services (?from=&to=)	OPERATOR	Servicios propios.
POST	/api/v1/operator/services/{bookingId}/start, /finish	OPERATOR	Inicia o finaliza un servicio.
GET	/api/v1/operator/ratings	OPERATOR	Calificaciones recibidas.
GET, POST	/api/v1/ratings/me, /api/v1/ratings/{bookingId}	CLIENT	Calificaciones dadas; califica una reserva (1 a 5, comentario de hasta 500).
GET	/api/v1/payment-accounts	Token	Cuentas activas con QR; la de efectivo aparece solo mientras está activa.
POST, GET	/api/v1/payments, /api/v1/payments/me	CLIENT	Reporta un pago con comprobante de máximo 2 MB (400 RECEIPT_TOO_LARGE); pagos propios.
GET	/api/v1/payments/me/bookings	CLIENT	Último estado de pago e indicador *payable* de cada reserva propia.
GET, POST	/api/v1/admin/payments (?status=)	ADMIN	Lista los pagos; registra un pago manual.
GET	/api/v1/admin/payments/amount-due	ADMIN	Monto real a pagar de una reserva (con el descuento del cupón).
POST	/api/v1/admin/payments/{id}/approve, /reject, /refund	ADMIN	Revisión y reembolso de un pago.
GET, POST, PUT	/api/v1/admin/payment-accounts[/{id}], /api/v1/admin/payment-methods	ADMIN	Cuentas y métodos de pago.
GET, POST, PUT, PATCH, DELETE	/api/v1/admin/promotions[/{id}[/active]], /api/v1/admin/promotions/metrics	ADMIN	Promociones (cupones), pausa y reanudación, y métricas.
GET	/api/v1/loyalty/balance, /api/v1/loyalty/promotions	CLIENT	Saldo de puntos y promociones con su indicador *unlocked*.
POST	/api/v1/loyalty/redeem	CLIENT	Canjea un cupón en una reserva.
GET	/api/v1/notifications, /api/v1/notifications/unread-count	Token	Bandeja (filtros y paginación) y contador.
PATCH, POST, DELETE	/api/v1/notifications/{id}/read, /unread, /read-all, /{id}	Token	Estado de lectura y borrado.
PUT, POST	/api/v1/notifications/devices, /devices/unregister	Token	Tokens push de Expo.
POST	/api/v1/admin/notifications	ADMIN	Notificación manual; solo SYSTEM_MESSAGE o INSPECTION_REPORT (400 NOTIFICATION_TYPE_NOT_ALLOWED). Ninguna pantalla de la web ni de la app la usa.

Método	Ruta	Rol	Descripción
GET	/internal/v1/users/{userId}/contact	Llave interna	Contacto e interruptores de notificación de un usuario para notification-service.
GET	/internal/v1/users/admins	Llave interna	Identificadores de los administradores activos.
GET	/internal/v1/bookings?from=&to=, /internal/v1/bookings/{id}	Llave interna	Reservas para la asignación automática de operations-service.
POST	/internal/v1/bookings/{id}/no-show	Llave interna	Pasa a NO_SHOW una reserva confirmada que terminó sin operario; solo la llama operations-service (409 BOOKING_NOT_EXPIRED si todavía no vence).
POST	/internal/v1/notifications/app-update	Llave interna	Aviso push de una versión nueva de la aplicación móvil.
GET	/internal/v1/payments/bookings/{bookingId}/reported	Llave interna	Desde el 26 de octubre de 2026, payment-service: si la reserva tiene un pago IN_REVIEW o APPROVED (*{"bookingId":123,"reported":true}*); solo la llama booking-service antes de cancelar por falta de pago.
GET	/actuator/health	Público	Salud de cada servicio y del gateway.

Nota. Elaborado por los autores a partir de los controladores de los servicios y de la documentación de los servicios.

13.3.1 Creación de una Reserva

El cuerpo de la solicitud usa fechas locales del lavadero. El servidor calcula la hora de finalización, la bahía, el precio y los puntos.

```
POST /api/v1/bookings
Authorization: Bearer <JWT>
Content-Type: application/json

{
  "vehicleId": 12,
  "serviceIds": [1],
  "date": "2026-10-12",
  "time": "10:30",
  "notes": "Revisar la tapicería",
  "paymentMode": "DIGITAL"
}
```

Nota. Estructura de *CreateBookingRequest* de la aplicación web; los valores son ficticios.

Tabla 21

Respuestas de Error Relevantes al Crear una Reserva

Situación	Respuesta
El vehículo no existe o pertenece a otro cliente	404 VEHICLE_NOT_FOUND (la misma respuesta a propósito).
customer-service no responde	503 CUSTOMER_SERVICE_UNAVAILABLE.
La hora no se puede reservar	409 SLOT_UNAVAILABLE con hasta cinco horas alternativas y el motivo: CLOSED_DAY, OUTSIDE_HOURS, TIME_PASSED o NO_FREE_BAY.
El vehículo ya tiene una reserva a esa hora	409 VEHICLE_ALREADY_BOOKED.
El servicio no se ofrece para el tipo de vehículo	SERVICE_NOT_AVAILABLE_FOR_VEHICLE.
Fecha no válida (pasada o más allá de 60 días)	INVALID_DATE.
Forma no válida de la solicitud	400 VALIDATION_ERROR.
El cliente no envió *paymentMode* o no es *DIGITAL* ni *CASH* (desde el 26 de octubre de 2026)	400 INVALID_PAYMENT_MODE.
El cliente ya tiene *BOOKING_CASH_ACTIVE_LIMIT* reservas activas en efectivo	409 CASH_BOOKING_LIMIT.

Nota. Códigos tomados del archivo de traducción de la aplicación web y de la documentación de booking-service.

13.4 Eventos de Dominio (RabbitMQ)

La mensajería usa un solo exchange de tipo topic, *carwash.events*. La clave de enrutamiento sigue la convención `<esquema-dueño>.<evento_en_snake_case>` (Hohpe y Woolf, 2003). Todo mensaje lleva el mismo sobre:

```
{
  "eventId": "uuid (clave de idempotencia)",
  "eventType": "BookingConfirmed",
  "aggregateId": "145",
  "occurredAt": "2026-10-12T15:00:00Z",
  "version": 1,
  "payload": { "bookingId": 145, "customerUserId": 7, "scheduledStart": "2026-10-12T15:00:00Z" }
}
```

Tabla 22

Eventos Publicados y Consumidos

Evento	Clave de enrutamiento	Publicador	Consumidores	Estado
UserRegistered	security.user_registered	security-service	customer-service, notification-service	Publicado
BookingConfirmed	booking.confirmed	booking-service	notification-service, operations-service (asignación automática)	Publicado
BookingCancelled	booking.cancelled	booking-service	notification-service	Publicado; lleva *reason* y, desde el 24 de octubre de 2026, *reasonCode*
BookingStatusChanged	booking.status_changed	booking-service	notification-service	Publicado cuando el administrador cambia el estado (desde el 23 de octubre de 2026); con IN_PROGRESS, COMPLETED o NO_SHOW cancela los recordatorios y NO_SHOW tiene su aviso (*BOOKING_NO_SHOW*, desde el 24 de octubre)
OperatorAssigned	execution.operator_assigned	operations-service	notification-service	Publicado
OperatorUnassigned	execution.operator_unassigned	operations-service	notification-service	Publicado desde el 24 de octubre de 2026 cuando una reserva reprogramada queda sin operario libre: avisa al operario liberado y a los administradores
ServiceStarted	execution.service_started	operations-service	notification-service	Publicado
ServiceCompleted	execution.service_completed	operations-service	notification-service	Publicado
InspectionShared	execution.inspection_shared	operations-service	notification-service	Publicado (con el token del enlace público)
ServiceNotStarted, ServiceNotFinished	execution.service_not_started, execution.service_not_finished	operations-service	notification-service	Publicado una vez por reserva (desde el 21 de octubre de 2026)
PaymentReported	payment.reported	payment-service	booking-service	Publicado desde el 26 de octubre de 2026, al reportar un pago con comprobante
PaymentConfirmed	payment.confirmed	payment-service	notification-service; booking-service desde el 26 de octubre de 2026	Publicado
PaymentRejected, PaymentRefunded	payment.rejected, payment.refunded	payment-service	notification-service	Publicado
PromotionRedeemed	payment.promotion_redeemed	payment-service	notification-service	Publicado

Evento	Clave de enrutamiento	Publicador	Consumidores	Estado
LoyaltyPointsEarned	payment.loyalty_points_earned	payment-service	notification-service	Publicado (con unlockedPromotions)
BookingCreated	booking.created	—	—	No publicado; desde el 24 de octubre de 2026 notification-service ya no lo escucha (H-12)

Nota. La entrega es al menos una vez; confirmación después del procesamiento; tres reintentos y luego la cola de mensajes muertos (*.dlq*); publicación después del commit, sin Outbox. Desde el 24 de octubre de 2026 operations-service consume de *operations-service.booking-confirmed* con su *.dlq* (la cola vieja *operations-service.events* se vacía y se borra al arrancar) y publica con confirmación del broker; notification-service guarda en la tabla *pending_delivery*, en la misma transacción, el correo y la push que envía después del commit, y los reintenta (1, 2, 4… minutos, máximo 1 hora, hasta 5 intentos). Cada evento que genera una notificación lleva el identificador de usuario del destinatario (*customerUserId* u *operatorUserId*), de modo que notification-service no consulta otros servicios (ADR-011). Desde el 26 de octubre de 2026 booking-service consume de *booking.payment-events* (llaves *payment.reported* y *payment.confirmed*), con su cola de mensajes muertos *booking.payment-events.dlq* por el exchange *carwash.events.dlx*. Como no hay Outbox, un *payment.reported* puede perderse; por eso *PaymentDeadlineJob* confirma con payment-service antes de cancelar (AC-89).

13.5 Salud y Verificación

Cada servicio y el gateway exponen */actuator/health* (y payment-service también */health*). Todos comprueban la base de datos: payment-service espera como máximo 3 segundos a SQL Server y, si no responde, contesta DOWN con el código 503. En el servidor público, *docker-compose.prod.yml* revisa además la salud de todos los contenedores cada 30 segundos (*docker compose ps* los muestra *healthy* o *unhealthy*). Las verificaciones de integración recomendadas se describen en el Manual de Instalación, sección 16, y las pruebas de punta a punta de *lavarapido-infra/e2e* recorren los flujos completos contra los contenedores; corren solo en local, porque el CI solo revisa su sintaxis.

14. Seguridad

La política de seguridad del proyecto se basa en la defensa en profundidad, el mínimo privilegio, los valores seguros por defecto, la seguridad desde el diseño y la confianza cero, y su lista de verificación sigue el OWASP Top 10 (OWASP Foundation, 2021).

14.1 Autenticación y Control de Sesiones

Tabla 23

Parámetros de Autenticación

Elemento	Decisión
Token	JWT Bearer emitido por security-service.
Algoritmo	HS256 con un secreto compartido (mínimo 32 bytes); RS256 es la evolución planeada.
Vigencia	Una hora (JWT_ACCESS_TOKEN_TTL); sin token de actualización.
Claims	sub, roles, sid, iss, aud, iat, nbf, exp y jti; sin nombre, correo, teléfono ni documento.
Contraseñas	BCrypt con costo 12 (Provos y Mazières, 1999); máximo 72 caracteres, el límite de entrada de BCrypt.
Intentos de inicio de sesión	Cinco intentos fallidos en 15 minutos bloquean la cuenta durante 15 minutos (423 ACCOUNT_LOCKED, tabla *security.login_lockout*); un ingreso correcto o el restablecimiento de la contraseña limpian el contador; el tiempo de respuesta es el mismo para los correos desconocidos.
Sesiones	security.user_session registra cada ingreso; el cierre de sesión fija revoked_at, y desactivar o inhabilitar la cuenta cierra todas sus sesiones.
Revocación	Desde el 21 de octubre de 2026 el token de una sesión cerrada deja de servir: security-service lo rechaza por su claim *sid* (*ActiveSessionValidator*) y el gateway lo corta antes de los demás servicios (*SessionCheckFilter*, respuesta guardada 30 segundos por token; si security-service no responde, la petición sigue: riesgo aceptado D-02). Desde el 24 de octubre de 2026 el filtro revisa toda petición con *Authorization*, con cualquier esquema (*bearer*, *Bearer*, *BEARER*), y el cierre de sesión borra la caché en todas sus variantes (SEC-01).

Nota. Fuente: configuración y código de security-service y ADR-006.

14.2 Autorización

El sistema tiene tres roles fijos (ADMIN, OPERATOR y CLIENT) y no tiene roles personalizados; las tablas *permission* y *role_permission* guardan una matriz informativa de permisos que edita el administrador, pero ninguna ruta la comprueba todavía (ADR-015). Cada servicio declara en su configuración de seguridad qué roles llegan a cada ruta (por ejemplo, */api/v1/admin/** exige ADMIN). Un rol válido no basta: cada servicio también verifica la **propiedad**, de modo que un operario no puede iniciar el servicio de otro y un cliente solo llega a sus propias reservas, vehículos, pagos, puntos y cupones. Las llamadas entre servicios reenvían el token del usuario, de modo que el servicio destino autoriza al usuario original. Las rutas */internal/** usan un filtro distinto (*InternalApiKeyFilter*) con la llave *INTERNAL_API_KEY*; si la llave está vacía, las rutas permanecen cerradas.

14.3 Validación de Entradas

- Validación en el borde con Bean Validation (Java) y en el dominio con objetos de valor (EmailAddress, DocumentNumber, PhoneNumber, LicensePlate, PersonName); las validaciones de los formularios son solo una comodidad.
- Listas permitidas en lugar de listas bloqueadas (por ejemplo, correos @gmail.com, formatos de placa colombiana, cuatro temas, imágenes PNG o JPG y solo dos tipos de notificación manual).
- **Inyección SQL:** JPA y EF Core con parámetros; está prohibido concatenar SQL.
- **XSS:** la interpolación de Angular escapa por defecto; no se omite el sanitizador.
- Los precios, los totales, los descuentos y las horas de finalización nunca vienen del cliente.

14.4 Secretos y Comunicación

- Los secretos (contraseña de sa, secreto JWT, SMTP, RabbitMQ y llave interna) se leen de variables de entorno o de *lavarapido-infra/.env*, excluido de Git; el repositorio solo contiene *.env.example*.
- En el entorno local los servicios se conectan con *sa*; antes de cualquier despliegue compartido, cada servicio debe tener un inicio de sesión con derechos solo sobre sus esquemas.

- HTTPS con TLS 1.2 o superior es obligatorio fuera del entorno local; solo se debe exponer el gateway. En el servidor público, Caddy agrega desde el 20 de octubre de 2026 los encabezados HSTS, *X-Content-Type-Options: nosniff*, *X-Frame-Options*, *Referrer-Policy* y *Permissions-Policy*, quita el encabezado *Server* y envía con la web una política de contenido (CSP) que solo permite código propio, Google Fonts y jsDelivr.
- Las cuentas de demostración de *DevDataSeeder* existen solo en el perfil *dev*.

14.5 Lista de Verificación OWASP Top 10

Tabla 24

Controles por Categoría de OWASP

Categoría	Control	Estado
A01 Pérdida de control de acceso	Rol por ruta y propiedad en cada servicio; identidad tomada del token.	Aplicado
A02 Fallas criptográficas	BCrypt; secretos en variables de entorno; TLS fuera del entorno local.	Aplicado (TLS con Let's Encrypt en el servidor público)
A03 Inyección	Consultas parametrizadas; validación de entradas.	Aplicado
A04 Diseño inseguro	Misma respuesta para correos desconocidos; los identificadores no se usan como control de acceso.	Aplicado
A05 Configuración de seguridad incorrecta	Sin trazas de pila en las respuestas; Swagger solo en dev; encabezados de seguridad y CSP en el servidor público.	Aplicado
A06 Componentes vulnerables	Revisión con versions:display-dependency-updates, dotnet list package --vulnerable y npm audit.	Manual
A07 Fallas de identificación y autenticación	Token de una hora; el token de una sesión cerrada deja de servir con cualquier esquema; bloqueo temporal por intentos fallidos, también del código de recuperación y de la contraseña actual; límite de 60 peticiones por minuto por IP en */api/v1/auth/** con la IP real del cliente (desde el 24 de octubre de 2026); consentimiento exigido en el registro.	Aplicado
A08 Fallas de integridad del software y de los datos	Versiones fijas; validación de los mensajes; mensajes no válidos a la DLQ.	Aplicado
A09 Fallas de registro y monitoreo de seguridad	Registros sin datos personales y con identificador de correlación; auditoría no poblada.	Parcial
A10 Falsificación de solicitudes del lado del servidor	Los servicios solo llaman a direcciones internas configuradas.	Aplicado

Nota. Elaborado por los autores a partir de la política de seguridad y del código.

14.6 Protección de Datos Personales

El tratamiento de datos personales debe cumplir la Ley Estatutaria 1581 de 2012 (Congreso de la República de Colombia, 2012). El registro exige la aceptación explícita de los Términos y Condiciones y de la Política de Tratamiento de Datos Personales (dos casillas en la web y en el móvil). *RegisterRequest* envía *acceptsTerms* y *acceptsDataPolicy*, y security-service guarda la fecha y la hora de cada aceptación en *security.app_user* (columnas *terms_accepted_at* y *data_policy_accepted_at*, changelog 039). No se recogen datos de tarjetas. Los únicos datos personales que viajan por el bus de mensajes son el correo y el nombre en UserRegistered, y su *toString()* los omite para que no lleguen a los registros.

14.7 Riesgos Aceptados

- El token de una sesión cerrada todavía sirve hasta 30 segundos (la respuesta que guarda el gateway) o, si security-service no responde, hasta que vence.

- Firma simétrica HS256 con un secreto compartido entre servicios.
- El límite de solicitudes cubre solo */api/v1/auth/**, vive en la memoria del gateway (por réplica) y no agrupa IPv6 por /64; el resto de la API no tiene límite.
- Posible pérdida de eventos entre el commit y la publicación (sin Outbox, TD-011).
- Instancia compartida de SQL Server y cuenta *sa* en el entorno local.
- Permisos por rol editables pero no aplicados (ADR-015).
- Campos de presentación y de descuento de una promoción que ninguna restricción mantiene coherentes (ADR-015).

15. Control de Versiones

15.1 Código Fuente

El proyecto usa Git con repositorios en GitHub. Se trabaja en ramas *feature/**, *feat/** o *fix/** que se integran en *develop* (integración); *main* guarda las versiones estables. El 23 de octubre de 2026 se pasó *develop* a *main* y se etiquetó *v1.0.0* en 8 repositorios (web, api-gateway, security, customer, booking, operations, notification e infra); payment-service la tuvo el 24 de octubre (PR #2 fusionado), y ese mismo día *develop* volvió a avanzar con los arreglos de la auditoría. Esos arreglos salieron el 26 de octubre de 2026 en *v1.1.0*, en los 10 repositorios, incluida la app móvil, cuya primera versión es esta (PR #1 «Release v1.1.0»). Ese mismo día salió *v1.1.1* en la web y la app móvil (términos y condiciones en tercera persona, con el apartado del informe de inspección y sus fotografías), en payment-service (conteo de pruebas del README) y en infra (payment-service se despliega desde *main*, como los demás). Los commits siguen el formato *tipo(alcance): descripción* (por ejemplo, *feat(loyalty): publish the points earned and the coupons they unlock when a payment is approved*). Las convenciones están en *00-governance/git-conventions.md*.

15.2 Esquema de la Base de Datos

El esquema se versiona con Liquibase: cada cambio es un changeset con un identificador y un autor, y Liquibase guarda una suma de comprobación. Los changesets son inmutables y solo avanzan; un cambio posterior se registra en un changeset nuevo.

15.3 Decisiones de Arquitectura

Tabla 25

Registros de Decisiones de Arquitectura

ADR	Tema	Estado
ADR-001	Idioma de la documentación (inglés)	Aceptado
ADR-002	Estilo arquitectónico y descomposición original	Reemplazado por ADR-009
ADR-003	Estrategia de base de datos: una instancia, un dueño por esquema	Aceptado
ADR-004	Comunicación entre servicios: REST y RabbitMQ	Aceptado
ADR-005	API Gateway	Aceptado
ADR-006	Autenticación y autorización: JWT HS256 verificado por cada servicio	Aceptado
ADR-007	Arquitectura interna: hexagonal	Aceptado
ADR-008	Verificación de pagos por QR y comprobante	Aceptado
ADR-009	Descomposición revisada en seis servicios	Propuesto (en uso)
ADR-010	Alineación del modelo de datos y el frontend	Aceptado
ADR-011	Notificaciones: push, eventos, correos y recordatorios	Aceptado
ADR-012	Reservas: resolución del vehículo por REST	Aceptado
ADR-013	payment-service en .NET 8 con arquitectura por capas	Propuesto (en uso)
ADR-014	Operaciones: integración con reservas, aprovisionamiento de operarios y asignación manual	Propuesto (en uso)
ADR-015	Puntos de fidelización, promociones, permisos de los roles fijos y bahías por reserva	Aceptado

Nota. Fuente: carpeta *05-architecture/decisions/records* de la documentación.

15.4 Integración y Despliegue Continuos

Integración continua. Cada uno de los 10 repositorios de código tiene *.github/workflows/ci.yml*, que corre en cada push o pull request a *develop* y *main*: *./mvnw verify* en los servicios Java y el gateway, *dotnet build* y *dotnet test* en payment-service, la compilación de producción y *ng test* en la web, *tsc --noEmit* y *npm test -- --ci* en la app móvil, y *shellcheck*, *actionlint*, *docker compose config* y la sintaxis de *e2e/* y *e2e/remote/* en *lavarapido-infra*. Al 24 de octubre de 2026 la última corrida de *develop* está en verde en los 10. La rama *main* tiene protección (revisión obligatoria y CI) en los repositorios de Brayan Cabrera, customer y booking; payment-service y la app móvil no tienen protección y *develop* no está protegida en ninguno.

Despliegue. El servidor de Azure se actualiza con *deploy/update-server.sh* (respaldo, *git pull* de cada repositorio, *docker compose up -d --build*). Desde el 24 de octubre de 2026, antes de reconstruir guarda la versión que corre como *:prev*, después espera hasta 8 minutos a que los 11 contenedores queden sanos, reinicia Caddy si cambió su *Caddyfile* y, si todo quedó sano, limpia las imágenes sin uso; *deploy/rollback.sh* vuelve a *:prev* sin compilar, y *setup-server.sh* nunca baja de versión (*merge-base --is-ancestor*; *ALLOW_DOWNGRADE=1* lo permite a propósito). Ese día se desplegó *develop* en dos corridas por SSH (la segunda con *BRANCH=develop*, que hizo falta hasta *v1.1.0*): 11 contenedores sanos y las migraciones 044, 045, 113, 047, 116, 022 y 023 aplicadas (Documento de Despliegue, sección 1.3.9). En *main* (*v1.0.0*) también está el flujo manual "Desplegar en Azure" (*deploy.yml*), que entra por SSH con una llave limitada a *deploy/ci-deploy.sh*; al 24 de octubre de 2026 no se ha ejecutado nunca y el repositorio no tiene sus secretos.

Despliegue continuo (fusionado en *develop* el 24 de octubre de 2026, sin secretos). *lavarapido-infra* (*006b5b0*) y el job "Pedir despliegue" de los 8 repositorios que se despliegan en Azure (web, api-gateway, security, customer, booking, operations, notification y payment) hacen que cada fusión aprobada a *main* despliegue sola: el CI del repositorio, si pasa, lanza *deploy.yml* con *automatic=true* (secreto *DEPLOY_DISPATCH_TOKEN*); el flujo respalda, baja *main* de todos los repositorios (hasta *v1.1.0*, payment-service se quedaba en *develop* por *DEVELOP_ONLY*; desde *v1.1.1* de infra también baja *main*), espera a otra actualización en curso hasta 20 minutos, exige que todos los contenedores queden sanos en 8 minutos, reinicia Caddy si cambió su *Caddyfile* y comprueba la web y la API. Con la máquina apagada deja un aviso y no falla; al encenderla, *deploy/boot-update.sh* (tarea *@reboot*) despliega lo pendiente comparando con *~/lavarapido/backups/deployed-heads*. La app móvil tiene *cd.yml*: con cada push a *main* repite el CI, compara la huella nativa del código con la del APK publicado y, si coincide, publica la actualización por internet (*eas update*, secreto *EXPO_TOKEN*); si cambió, pide un APK nuevo. Activarlo exige crear *AZURE_SSH_KEY* y *DEPLOY_DISPATCH_TOKEN* (en customer, booking y payment los crea su dueño o un administrador) y *EXPO_TOKEN*, (*v1.1.0* ya se liberó el 26 de octubre de 2026, con *lavarapido-infra* al final; Documento de Despliegue, sección 4.3.4). Quedó cerrado como AC-90 en el documento de acciones correctivas. **Primera corrida automática (26 de octubre de 2026, noche, hora de Colombia).** Con los secretos ya creados, la fusión del PR «Release v1.1.1» a *main* de *lavarapido-infra* lanzó a las 22:19 el flujo **Desplegar en Azure** con *automatic=true*, y los pedidos siguientes de los demás repositorios desplegaron solos hasta las 22:48 (los que llegaron mientras otro corría se cancelaron y los atendió el siguiente): todos terminaron en verde, con «Listo: todos los servicios están sanos» y «Web y API responden 200». Desde la corrida de las 22:46, payment-service también baja *main*. En la app móvil, la publicación de *v1.1.0* falló porque faltaba *EXPO_TOKEN* (no se publicó nada); con el secreto creado, el APK *versionCode* 6 se publicó a las 22:39 y la corrida de *v1.1.1* comparó la huella nativa del commit con la de ese APK, vio que coincidía y publicó la actualización por internet (Documento de

Despliegue, sección 1.3.12). **SonarCloud (26 de octubre de 2026).** Se corrigieron los hallazgos del análisis: en api-gateway, un acceso nulo real en la ruta de autenticación y la comparación de las redes CIDR por contenido; en security, operations y notification, los publicadores de RabbitMQ y los remitentes SMTP y Expo marcados como *@Primary* (regla *java:S9352*); en la web, el CI más estricto, la CLI de Angular fijada y el orden con *localeCompare*; y en la app móvil, promesas marcadas con *void*, *callbacks* envueltos en funciones flecha, el componente *Pill* compartido, la clave de traducción de la contraseña renombrada y los scripts y flujos del CI endurecidos.

16. Recomendaciones de Mantenimiento

16.1 Entorno Local

El entorno local y su orden de arranque se describen en el Manual de Instalación. SQL Server y RabbitMQ deben estar sanos antes de los servicios; cada servicio debe tolerar que una dependencia no esté lista y responder 503 con un código estable.

16.2 Configuración

Las variables de configuración se describen en el Manual de Instalación, sección 10. Toda variable nueva se debe agregar a *.env.example* con un comentario y, si el servicio corre en Docker, a su bloque *environment* de *docker-compose.yml*. Las del 21 de octubre de 2026 (*SESSION_CHECK_** del gateway, *BOOKING_EXPIRY_** de booking-service, con *BOOKING_EXPIRY_GRACE_MINUTES* compartida con operations-service, y *FOLLOW_UP_ENABLED*, *FOLLOW_UP_INTERVAL* y *FOLLOW_UP_LOOK_BACK_DAYS* de operations-service; nueve en total) ya están en esos archivos (acción ACO-14, cerrada). Las del 23 de octubre de 2026 (*ALLOWED_EMAIL_DOMAINS* y *DEMO_*_PASSWORD* de security-service) también. *ADMIN_DOCUMENT* y *ADMIN_PHONE* completan los datos del primer administrador en producción. Las del 26 de octubre de 2026 (*BOOKING_CASH_ACTIVE_LIMIT*, *BOOKING_PAYMENT_WINDOW*, *BOOKING_PAYMENT_EXPIRY_ENABLED*, *BOOKING_PAYMENT_EXPIRY_INTERVAL* y *PAYMENT_SERVICE_URL* de booking-service, e *INTERNAL_API_KEY*, que ahora también necesita payment-service y que *docker-compose.prod.yml* exige) están en *.env.example* y en el compose.

16.3 Actualizaciones y Dependencias

- Antes de cada entrega, ejecutar *./mvnw versions:display-dependency-updates*, *dotnet list package --outdated --vulnerable*, *npm outdated* y *npm audit*, y corregir las vulnerabilidades críticas y altas.
- Fijar versiones exactas; no usar versiones flotantes.
- Mantener actualizados los ADR, los modelos de datos y los contratos OpenAPI cuando cambie el diseño; exportar los contratos desde el servicio en ejecución (*/v3/api-docs*).

16.4 Base de Datos

- Una tabla de dominio nueva debe crear su disparador *tr_touch_* en el mismo changeset.
- Toda consulta nueva debe incluir *deleted_at IS NULL*.
- Las semillas usan identificadores fijos; nunca volver a sembrar sin *IDENTITY_INSERT* (CANCELLED = 5, APPROVED = 3).
- El siguiente changeset de la numeración global es el 050, y la siguiente semilla, la 118 (al 26 de octubre de 2026); payment-service va en su 023.

16.5 Pruebas

La estrategia de pruebas propone una pirámide de pruebas unitarias, de integración, de contrato y de punta a punta, con al menos un 80 % de cobertura de líneas y un 90 % en el dominio. Al 24 de octubre de 2026, el último CI de *develop* de cada repositorio

aprobó 1 119 pruebas: web 212 en 66 archivos, gateway 62, security-service 200, customer-service 82, booking-service 95, operations-service 110, notification-service 133 y payment-service 225 (134 de dominio, 52 de aplicación, 34 de la API y 5 de infraestructura; las 11 de persistencia se omiten en el CI porque necesitan la base aislada *LavaRapidoPaymentIT*); la app móvil tiene 73 pruebas en 25 suites con *jest-expo*: 1 192 en total. *lavarapido-infra/e2e/remote* agrega una batería de 44 casos contra el servidor público (*e2e.yml*, a mano), probada solo contra un simulador. El 23 de octubre de 2026 eran 838. La ejecución completa anterior, en la noche del 21 de octubre de 2026, dio estos resultados: 707 pruebas unitarias y de integración aprobadas, ninguna omitida (web 62 en 41 archivos, gateway 40, security-service 155, customer-service 69, booking-service 50, operations-service 67, notification-service 114 y payment-service 150: 115 de dominio, 28 de aplicación y 7 de persistencia contra la base aislada *LavaRapidoPaymentIT*), 18 más que en la ejecución anterior del mismo día por las clases nuevas *FollowUpPolicyTest* y *FollowUpServiceTest* (operations) y *DomainEventEnvelopeTest* (notification), la ruta interna de no asistió en *BookingServiceTest* y *BookingApiWebTest* y "Por cerrar" en *reservations-store.spec.ts*; además, 19 de 19 pruebas de punta a punta, ya con el chequeo de sesión del gateway. Hasta el 23 de octubre de 2026 la app móvil no tenía pruebas automatizadas y su integración continua solo revisaba los tipos (*npx tsc --noEmit*); los 42 casos funcionales se ejecutaron por última vez el 19 de octubre de 2026 y pasaron todos contra el sistema completo (*lavarapido-infra/e2e*: flujo completo de reserva, pago y servicio, inspección y reglas de acceso). Los servicios Java prueban su dominio, sus casos de uso y su capa web sin base de datos (*./mvnw test*); payment-service tiene tres proyectos de prueba (dominio, aplicación e integración). Todavía no hay pruebas de persistencia con un SQL Server real dentro de los servicios Java (TD-010); las pruebas de punta a punta cubren en parte ese hueco. Las pruebas de concurrencia recomendadas son dos reservas simultáneas para la misma bahía, dos actualizaciones con la misma *row_version*, dos movimientos de puntos simultáneos del mismo cliente y la entrega duplicada del mismo evento.

16.6 Diagnóstico de Problemas Frecuentes

Tabla 26

Problemas Frecuentes y Verificaciones Sugeridas

Síntoma	Causa probable	Verificación
401 con un token que parecía válido	Token vencido, JWT_SECRET, emisor o audiencia distintos, o sesión cerrada (logout en otro dispositivo o cuenta inhabilitada).	Ingresar de nuevo; verificar el archivo .env; *GET /api/v1/auth/session* con el token dice si la sesión sigue abierta.
400 RECEIPT_TOO_LARGE al reportar un pago	El comprobante pesa más de 2 MB (un cliente sin la compresión de la web o de la app actualizada).	Subirlo desde la web o desde la app actualizada, que comprimen la imagen.
Una reserva pasó sola a No asistió o Cancelada	Pasó una hora de su fin y era programada sin confirmar (*ExpiredBookingsJob*) o confirmada sin operario (*FollowUpJob*).	Estado y motivo de la reserva (EXPIRED en las canceladas); registros de booking-service (*Closed N expired bookings*) y de operations-service (*Follow-up: N booking(s) closed as not attended*).
Una reserva aparece en "Por cerrar"	Tenía operario y nadie la inició, o se inició y nadie la finalizó.	*GET /api/v1/admin/assignments/to-close*; estado de sus filas de *service_execution* y *follow_up_notice_at*.
503 CUSTOMER_SERVICE_UNAVAILABLE o BOOKING_SERVICE_UNAVAILABLE	La dependencia está caída o su URL es incorrecta.	Salud del servicio y CUSTOMER_SERVICE_URL o BOOKING_SERVICE_URL.
"Perfil de cliente no listo" (CUSTOMER_NOT_PROVISIONED)	No se consumió el evento UserRegistered.	Cola customer-service.events y su DLQ.
Notificaciones que no llegan	MESSAGING_ENABLED=false, un evento sin destinatario o un mensaje en la DLQ.	Configuración y cola notification-service.events.dlq.
La construcción de Compose falla para operations-service	El repositorio no se clonó como lavarapido-operation-service.	Manual de Instalación, sección 8.
No se acreditan puntos después de una aprobación	Servicios con 0 puntos, MESSAGING_ENABLED=false (sin aviso) o BOOKING_SERVICE_URL incorrecta.	payment.loyalty_transaction y el registro de payment-service.
Push no entregado	PUSH_ENABLED=false, Expo Go, interruptor de push desactivado o un token desactivado.	Configuración, security.user_preference y notification.device_token.
Error de suma de comprobación de Liquibase	Se editó un changeset ejecutado.	Revertir y crear un changeset nuevo.

Nota. Elaborado por los autores a partir de los runbooks de los servicios.

16.7 Deuda Técnica

Tabla 27

Principales Elementos de Deuda Técnica

Elemento	Descripción	Prioridad
TD-007	Reenviar X-Correlation-Id de customer-service y notification-service a security-service y en los mensajes; generarlo en el gateway.	Baja
TD-008	Bloqueo optimista con @Version.	Baja
TD-010	Pruebas de persistencia con un SQL Server real (Testcontainers).	Baja
TD-011	Outbox para los eventos.	Baja
TD-012	Métricas con Micrometer y Prometheus.	Baja
TD-019	Changelogs de pagos con numeración propia.	Baja

Nota. Fuente: *15-project-control/technical-backlog.md* de la documentación (Equipo de Desarrollo Vehicle Washing, 2026a), revisado contra el código el 16 de octubre de 2026. Entre el 8 y el 16 de octubre de 2026 se resolvieron TD-001, TD-002, TD-003, TD-009, TD-013, TD-014, TD-015, TD-016, TD-017, TD-018, TD-020 y TD-021; su cierre se registra en el documento de acciones correctivas, preventivas y de mejoramiento. El 19 de octubre de 2026 se cerró también TD-006, con la integración continua en GitHub Actions. El 21 de octubre de 2026 se cerraron TD-024 (tokens de sesiones cerradas), TD-025 (reservas que nadie cerraba y conteo de reservas sin operario) y TD-026 (asignación doble de un operario); en la noche del mismo día se ajustó la regla de TD-025 para que una reserva con operario no quede en No asistió sin que lo decida el administrador; y TD-027 (pruebas de las pantallas del administrador) quedó resuelta para reservas, gestión y reportes.

16.8 Evolución Recomendada

Si el proyecto sale del alcance académico, la documentación identifica como evolución natural: firma RS256, tokens de actualización, límite de solicitudes en toda la API (hoy solo en */api/v1/auth/**), Outbox, un inicio de sesión de base de datos por servicio (o una base de datos por servicio), disparadores de auditoría, aplicación real de la matriz de permisos, mTLS entre servicios, observabilidad (métricas y trazabilidad), almacenamiento de imágenes fuera de la base de datos y una pasarela de pagos. Cada cambio requiere un ADR.

17. Glosario

Tabla 28

Glosario Técnico

Término	Definición
ADR	Registro de decisión de arquitectura.
Agregado	Grupo de objetos de dominio que se mantienen coherentes en conjunto, con una raíz (Evans, 2004); por ejemplo, Payment.
API Gateway	Punto de entrada externo único que enruta las solicitudes; en este sistema no verifica la firma de los tokens, pero desde el 21 de octubre de 2026 corta los de sesiones cerradas.
Arquitectura hexagonal	Estilo de puertos y adaptadores en el que el dominio no depende de la infraestructura (Cockburn, 2005).
Bahía	Puesto físico de lavado; el número de bahías activas es la capacidad.
Borrado lógico	Marcar *deleted_at* en lugar de borrar la fila.
Cola de mensajes muertos (DLQ)	Cola donde se guardan los mensajes que fallaron después de los reintentos.
Evento de dominio	Hecho del negocio que se publica para que otros servicios reaccionen.
Exchange de tipo topic	Exchange de RabbitMQ que enruta los mensajes por claves de enrutamiento.
Identificador de correlación	Identificador que sigue una solicitud a través del gateway, los servicios y los mensajes.
Idempotencia	Procesar el mismo evento más de una vez sin efectos duplicados.
JWT	JSON Web Token: token firmado que lleva la identidad y los roles (Jones et al., 2015).
Libro de puntos (ledger)	Registro de los movimientos de puntos de cada cliente; el saldo es el del último movimiento.
Liquibase	Herramienta para versionar y migrar el esquema de la base de datos.
Objeto de valor	Objeto definido por sus valores (por ejemplo, LicensePlate o EmailAddress).
Outbox	Patrón que guarda los eventos en la misma transacción antes de publicarlos; no adoptado.
Problem details	Formato estándar de los errores HTTP (RFC 9457) con un código estable.

Nota. Elaborado por los autores a partir del glosario del proyecto y de las referencias citadas.

Referencias

Cockburn, A. (2005). *Hexagonal architecture*. https://alistair.cockburn.us/hexagonal-architecture/

Congreso de la República de Colombia. (2012). *Ley Estatutaria 1581 de 2012, por la cual se dictan disposiciones generales para la protección de datos personales*. http://www.secretariasenado.gov.co/senado/basedoc/ley_1581_2012.html

Equipo de Desarrollo Vehicle Washing. (2026a). *Vehicle washing management system: Project documentation* (vehicle-w-docs) [Documentación de proyecto no publicada]. GitHub. https://github.com/code-sena/vehicle-w-docs

Equipo de Desarrollo Vehicle Washing. (2026b). *Front-end-proyecto-web* (rama develop) [Software]. GitHub. https://github.com/BrayanCABRERA123/Front-end-proyecto-web

Equipo de Desarrollo Vehicle Washing. (2026c). *lavarapido-customer-service* (rama develop) [Software]. GitHub. https://github.com/camiloguilombo12/lavarapido-customer-service

Equipo de Desarrollo Vehicle Washing. (2026d). *lavarapido-infra* (rama develop) [Software]. GitHub. https://github.com/BrayanCABRERA123/lavarapido-infra

Equipo de Desarrollo Vehicle Washing. (2026e). *lavarapido-payment-service* (rama develop) [Software]. GitHub. https://github.com/merari0322/lavarapido-payment-service

Equipo de Desarrollo Vehicle Washing. (2026f). *lavarapido-security-service* (rama develop) [Software]. GitHub. https://github.com/BrayanCABRERA123/lavarapido-security-service

Evans, E. (2004). *Domain-driven design: Tackling complexity in the heart of software*. Addison-Wesley.

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Tesis doctoral, University of California, Irvine]. https://ics.uci.edu/~fielding/pubs/dissertation/top.htm

Hohpe, G., y Woolf, B. (2003). *Enterprise integration patterns: Designing, building, and deploying messaging solutions*. Addison-Wesley.

Jones, M., Bradley, J., y Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc7519

Newman, S. (2021). *Building microservices: Designing fine-grained systems* (2.ª ed.). O'Reilly Media.

Nottingham, M., Wilde, E., y Dalal, S. (2023). *Problem details for HTTP APIs* (RFC 9457). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc9457

OWASP Foundation. (2021). *OWASP Top 10:2021*. https://owasp.org/Top10/

Provos, N., y Mazières, D. (1999). A future-adaptable password scheme. En *Proceedings of the 1999 USENIX Annual Technical Conference, FREENIX Track* (pp. 81–91). USENIX Association.

Servicio Nacional de Aprendizaje. (2026). *Guía para elaborar manuales de software: Manual de usuario, manual técnico, manual de instalación y manual de administrador* [Material de formación no publicado]. Programa Tecnólogo en Análisis y Desarrollo de Software.

18. Anexos

Anexo A. Origen de las Figuras

Las Figuras 1 y 3 son diagramas elaborados a partir del código. La Figura 2 se generó el 16 de octubre de 2026 desde la base de datos real, con todos los servicios arrancados, por lo que muestra las 57 tablas tal como existen. Ninguna figura contiene secretos ni datos personales.

Anexo B. Discrepancias Detectadas

La Tabla 29 reúne las contradicciones encontradas entre el código, la documentación del proyecto y las versiones anteriores de los manuales. Cada una se debe resolver antes de la entrega final.

Tabla 29

Discrepancias y Criterio Adoptado en Este Manual

Tema	Discrepancia	Criterio usado aquí
Pila de los servicios	La versión 2.0 indicaba Java 21 y Spring Boot 3 en los seis servicios.	Código: Spring Boot 4.0.8 en los servicios Java y .NET 8 en payment-service (ADR-013).
Migraciones	La versión 2.0 describía un solo changelog maestro para toda la base de datos.	Código y estrategia de migraciones: un changelog por servicio aplicado al arrancar; pagos mediante payment-migrate.
Gateway	La versión 2.0 decía que el gateway rechaza los JWT no válidos en el borde.	Documentación del gateway y ADR-006: no verifica la firma de los tokens; lo hace cada servicio. Desde el 2026-10-21 sí corta los tokens de sesiones cerradas, preguntando a security-service.
Bloqueo de cuenta	La versión 3.1 indicaba que no existía.	Código del 2026-10-16: cinco intentos fallidos en 15 minutos bloquean la cuenta 15 minutos (RF-004b). Resuelta.
Asignación de operarios	La versión 3.1 indicaba asignación solo manual y sin elección del cliente.	Código del 2026-10-16: asignación automática al confirmarse la reserva con la elección opcional del cliente (RF-008 y RF-010). Resuelta.
Cierre de reservas que nadie registró	La versión 4.5 decía que toda reserva CONFIRMED que nadie iniciaba pasaba sola a NO_SHOW una hora después de su fin.	Código de la noche del 2026-10-21: solo pasa sola la CONFIRMED sin operario; la que tenía operario queda "por cerrar" para el administrador (*FollowUpPolicy*). Resuelta.
Revocación de tokens	Hasta la versión 4.4 un token seguía sirviendo hasta su vencimiento después de cerrar sesión.	Código del 2026-10-21: el token de una sesión cerrada deja de servir en security-service y en el gateway (TD-024). Resuelta.
Auditoría	La arquitectura dice que los disparadores alimentan audit.audit_log.	Código: la tabla existe, pero ningún disparador ni servicio escribe en ella.
Solicitud de reserva	La versión 2.0 usaba customerVehicleId, scheduledStart y servicios con cantidad.	Código: vehicleId, serviceIds, date, time y notes.
Códigos de ejecución	La versión 2.0 usaba códigos en español (PENDIENTE, EN_PROCESO...).	Modelo de datos: PENDING, IN_PROGRESS, PAUSED, COMPLETED, WITH_ISSUE.
Número de tablas	La versión 2.0 citaba 43, 44 o 49 tablas; la versión 3.1 citaba 49.	57 tablas al 2026-10-16, comprobadas en la base de datos (se agregaron las del bloqueo de cuenta, la dirección de la persona, el perfil y las habilidades del operario y los cuatro de la inspección).
Formato de los errores	La versión 2.0 describía un objeto con error, message y traceId.	Código: problem details del RFC 9457 con code.
Promociones	La versión 3.0 indicaba que las promociones funcionaban solo en el navegador.	Código del 2026-10-09: cupones porcentuales en payment-service, desbloqueables con puntos (ADR-015). Resuelta.
Avisos de pago y puntos	La versión 3.0 indicaba que payment-service no publicaba eventos.	Código del 2026-10-09: publica cinco eventos y acredita puntos (ADR-011 y ADR-015). Resuelta.
Referencia repetida	ADR-008 y RF-016 prohíben reutilizar una referencia de transacción.	Código del 2026-10-09: se rechaza con TRANSACTION_REFERENCE_REUSED. Resuelta.
Nombre del repositorio de operaciones	La versión 3.0 de los manuales lo llamaba lavarapido-operations-service.	Nombre real: lavarapido-operation-service (el que espera docker-compose.yml). Resuelta.
Módulo de administración	La versión 2.0 del manual de administrador indicaba que no tenía código.	Código: módulo completo bajo /admin.
README de lavarapido-api-gateway	Decía que el gateway no respondía CORS y no listaba las rutas de operations-service y payment-service.	Corregido el 2026-10-21: responde CORS (*CorsConfig*), corta los tokens de sesiones cerradas y lista todas las rutas. Resuelta.
README de lavarapido-infra	Marcaba operations-service y payment-service como planeados.	Corregido el 2026-10-16: todos los servicios aparecen como implementados. Resuelta.
Guía del equipo (sección 6)	Listaba como datos de prueba pantallas que ya estaban conectadas.	Corregida el 2026-10-16: solo quedan los reportes, calculados en el navegador, y la matriz de permisos, que no se aplica. Resuelta.

Tema	Discrepancia	Criterio usado aquí
Textos de la interfaz	La página de bienvenida anunciaba un servicio "donde estés", el botón "Reportar progreso" no enviaba nada, la opción "Eliminar cuenta" solo desactivaba la cuenta y los textos del reembolso estaban solo en español.	Corregidos el 2026-10-16: la página habla del servicio en el lavadero, el botón se retiró, la opción se llama "Desactivar mi cuenta" y el reembolso está traducido a los cuatro idiomas. Resuelta.
Diccionario del esquema de notificaciones	Las columnas de device_token, processed_event y booking_reminder no se detallaron.	Propósito documentado; columnas por completar.

Nota. Elaborado por los autores a partir del análisis cruzado del código y de la documentación.

Anexo C. Lista de Verificación para la Entrega

Tabla 30

Lista de Verificación del Manual Técnico

Criterio	Cumple
Portada completa y versión	Sí / No
Tabla de contenido con páginas actualizadas	Sí / No
Objetivo y alcance explicados	Sí / No
El contenido coincide con el código de la versión final	Sí / No
Diagrama entidad-relación incluido	Sí / No
Estructura del proyecto verificada contra el código real	Sí / No
Discrepancias del Anexo B resueltas	Sí / No
Sin credenciales, llaves ni datos sensibles	Sí / No
Ortografía y formato APA revisados	Sí / No
Exportado a PDF con el nombre Manual_tecnico_LavadoVehicular_3145556.pdf	Sí / No

Nota. Adaptado de la lista de verificación de la guía para elaborar manuales de software (SENA, 2026).